

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



An-Najah-National-University

IDS PROJECT

Hybrid Multi-Interface IDS/IPS Platform Using Suricata

DR. Othman Othman

Made By:

Tamam Dreat (12220033)

Batool Qazaqi (12220116)

Shahad Tharefeh (12116503)

Hybrid Multi-Interface IDS/IPS Platform Using Suricata

Project Type	IDS/IPS Experimental Platform
Main Tool	Suricata
Testing Environment	VirtualBox, Monitoring Server, Kali Linux, Windows Client
Main Tests	LAN/WAN Monitoring, ICMP/HTTP Detection, Flood Detection, File Upload Analysis, Email Alerts, IP Blocking, Dashboard

This report documents the setup and results of a hybrid multi-interface IDS/IPS platform using Suricata, including LAN/WAN monitoring, ICMP/HTTP detection, behavioral analysis, email alerts, IPS blocking, and the final dashboard.

1. Experiment Overview

The experiment aims to build a hybrid monitoring platform that works across multiple network interfaces, separates LAN and WAN monitoring, analyzes logs, detects suspicious behaviors, and connects the detection layer with alerting and blocking capabilities.

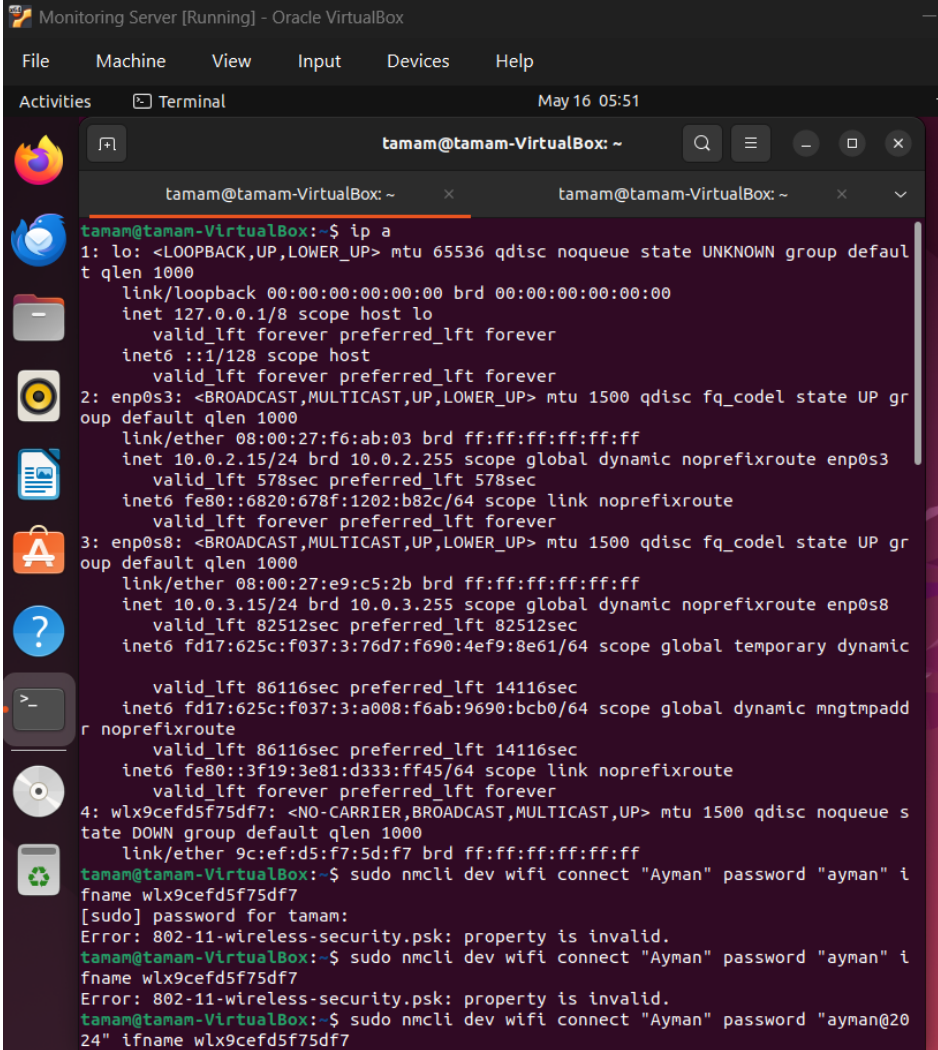
The platform uses custom Suricata rules, fast.log and global_behavior.log files, Python-based analysis scripts, iptables for blocking, and a Flask dashboard for result visualization.

2. Main Testing Phases

Phase 1	Network interface preparation and routing configuration.
Phase 2	Suricata custom rules for LAN and WAN traffic detection.
Phase 3	ICMP, HTTP, flood, and scanning traffic generation.
Phase 4	Behavior analyzer for file upload and threat scoring.
Phase 5	Email alerting and IPS blocking using iptables.
Phase 6	Threat intelligence dashboard and final visualization.

3. Network Interface and Routing Setup

Figure 1: Preparing the Wireless WAN Interface

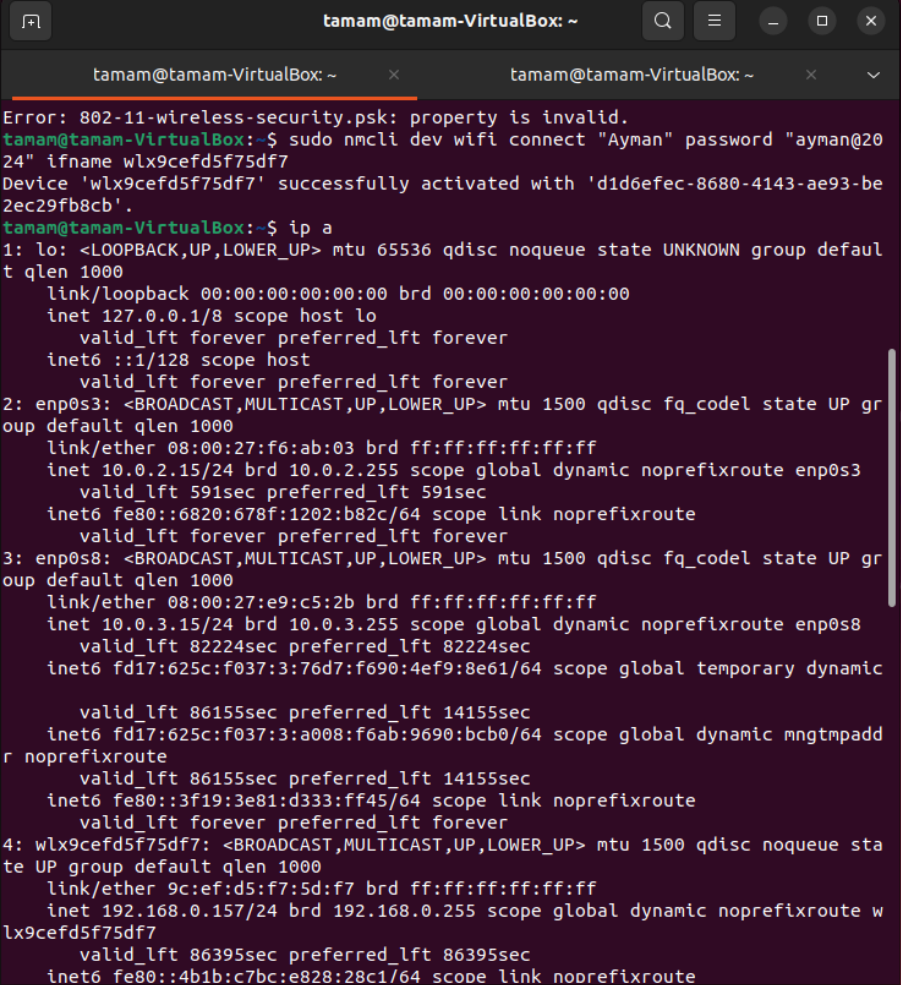


```

Monitoring Server [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal May 16 05:51
tamam@tamam-VirtualBox: ~
tamam@tamam-VirtualBox: ~
tamam@tamam-VirtualBox:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:f6:ab:03 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 578sec preferred_lft 578sec
    inet6 fe80::6820:678f:1202:b82c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:e9:c5:2b brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
        valid_lft 82512sec preferred_lft 82512sec
    inet6 fd17:625c:f037:3:76d7:f690:4ef9:8e61/64 scope global temporary dynamic
        valid_lft 86116sec preferred_lft 14116sec
    inet6 fd17:625c:f037:3:a008:f6ab:9690:bc0/64 scope global dynamic mngtmpdrd
        valid_lft 86116sec preferred_lft 14116sec
    inet6 fe80::3f19:3e81:d333:ff45/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: wlx9cefd5f75df7: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether 9c:ef:d5:f7:5d:f7 brd ff:ff:ff:ff:ff:ff
tamam@tamam-VirtualBox:~$ sudo nmcli dev wifi connect "Ayman" password "ayman" ifname wlx9cefd5f75df7
[sudo] password for tamam:
Error: 802-11-wireless-security.psk: property is invalid.
tamam@tamam-VirtualBox:~$ sudo nmcli dev wifi connect "Ayman" password "ayman" ifname wlx9cefd5f75df7
Error: 802-11-wireless-security.psk: property is invalid.
tamam@tamam-VirtualBox:~$ sudo nmcli dev wifi connect "Ayman" password "ayman@2024" ifname wlx9cefd5f75df7

```

The monitoring server interfaces are displayed, with an attempt to connect the wireless interface `wlx9cefd5f75df7` for later use as the WAN interface.

Figure 2: WAN Interface Activated Successfully


```

tamam@tamam-VirtualBox: ~
Error: 802-11-wireless-security.psk: property is invalid.
tamam@tamam-VirtualBox:~$ sudo nmcli dev wifi connect "Ayman" password "ayman@2024" ifname wlx9cefd5f75df7
Device 'wlx9cefd5f75df7' successfully activated with 'd1d6efec-8680-4143-ae93-be2ec29fb8cb'.
tamam@tamam-VirtualBox:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:f6:ab:03 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 591sec preferred_lft 591sec
    inet6 fe80::6820:678f:1202:b82c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:e9:c5:2b brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
        valid_lft 82224sec preferred_lft 82224sec
    inet6 fd17:625c:f037:3:76d7:f690:4ef9:8e61/64 scope global temporary dynamic
        valid_lft 86155sec preferred_lft 14155sec
    inet6 fd17:625c:f037:3:a008:f6ab:9690:bc0/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86155sec preferred_lft 14155sec
    inet6 fe80::3f19:3e81:d333:ff45/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: wlx9cefd5f75df7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 9c:ef:d5:f7:5d:f7 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.157/24 brd 192.168.0.255 scope global dynamic noprefixroute wlx9cefd5f75df7
        valid_lft 86395sec preferred_lft 86395sec
    inet6 fe80::4b1b:c7bc:e828:28c1/64 scope link noprefixroute
  
```

The wireless connection is activated and wlx9cefd5f75df7 receives 192.168.0.157, confirming that it is ready for monitoring.

Figure 3: Routing Configuration

```

te UP group default qlen 1000
    link/ether 9c:ef:d5:f7:5d:f7 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.157/24 brd 192.168.0.255 scope global dynamic noprefixroute w
lx9cefd5f75df7
    valid_lft 86395sec preferred_lft 86395sec
    inet6 fe80::4b1b:c7bc:e828:28c1/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
tamam@tamam-VirtualBox:~$ ip route
default via 10.0.2.1 dev enp0s3 proto dhcp metric 100
default via 10.0.3.2 dev enp0s8 proto dhcp metric 101
default via 192.168.0.1 dev wlx9cefd5f75df7 proto dhcp metric 600
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.3.0/24 dev enp0s8 proto kernel scope link src 10.0.3.15 metric 101
169.254.0.0/16 dev wlx9cefd5f75df7 scope link metric 1000
192.168.0.0/24 dev wlx9cefd5f75df7 proto kernel scope link src 192.168.0.157 met
ric 600
tamam@tamam-VirtualBox:~$ sudo ip route del default dev enp0s3
tamam@tamam-VirtualBox:~$ ip route
default via 10.0.3.2 dev enp0s8 proto dhcp metric 101
default via 192.168.0.1 dev wlx9cefd5f75df7 proto dhcp metric 600
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.3.0/24 dev enp0s8 proto kernel scope link src 10.0.3.15 metric 101
169.254.0.0/16 dev wlx9cefd5f75df7 scope link metric 1000
192.168.0.0/24 dev wlx9cefd5f75df7 proto kernel scope link src 192.168.0.157 met
ric 600
tamam@tamam-VirtualBox:~$ sudo ip route del default dev enp0s8
tamam@tamam-VirtualBox:~$ ip route
default via 192.168.0.1 dev wlx9cefd5f75df7 proto dhcp metric 600
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.3.0/24 dev enp0s8 proto kernel scope link src 10.0.3.15 metric 101
169.254.0.0/16 dev wlx9cefd5f75df7 scope link metric 1000
192.168.0.0/24 dev wlx9cefd5f75df7 proto kernel scope link src 192.168.0.157 met
ric 600
tamam@tamam-VirtualBox:~$ sudo ip route del default dev wlx9cefd5f75df7

```

The routing table is shown while old default routes are removed to force external traffic through the WAN interface.

Figure 4: ICMP Capture on WAN

```

tamam@tamam-VirtualBox:~$ sudo tcpdump -i wlx9cefd5f75df7 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wlx9cefd5f75df7, link-type EN10MB (Ethernet), snapshot length 26214
4 bytes
05:50:40.117545 IP tamam-VirtualBox > dns.google: ICMP echo request, id 1, seq 1
, length 64
05:50:40.230756 IP dns.google > tamam-VirtualBox: ICMP echo reply, id 1, seq 1,
length 64
05:50:41.118532 IP tamam-VirtualBox > dns.google: ICMP echo request, id 1, seq 2
, length 64
05:50:41.129923 IP dns.google > tamam-VirtualBox: ICMP echo reply, id 1, seq 2,
length 64
05:50:42.119936 IP tamam-VirtualBox > dns.google: ICMP echo request, id 1, seq 3
, length 64
05:50:42.130137 IP dns.google > tamam-VirtualBox: ICMP echo reply, id 1, seq 3,
length 64
05:50:43.121664 IP tamam-VirtualBox > dns.google: ICMP echo request, id 1, seq 4
, length 64
05:50:43.129255 IP dns.google > tamam-VirtualBox: ICMP echo reply, id 1, seq 4,
length 64
05:50:44.122601 IP tamam-VirtualBox > dns.google: ICMP echo request, id 1, seq 5
, length 64
05:50:44.131870 IP dns.google > tamam-VirtualBox: ICMP echo reply, id 1, seq 5,
length 64
05:50:45.123854 IP tamam-VirtualBox > dns.google: ICMP echo request, id 1, seq 6
, length 64
05:50:45.135639 IP dns.google > tamam-VirtualBox: ICMP echo reply, id 1, seq 6,
length 64

```

tcpdump captures ICMP requests and replies on the wireless interface, confirming that traffic is passing through the monitored WAN path.

Figure 5: External Connectivity Test

```

tamam@tamam-VirtualBox: ~
tamam@tamam-VirtualBox: ~ x tamam@tamam-VirtualBox: ~ x v
tamam@tamam-VirtualBox:~$ ping -4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=119 time=113 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=119 time=11.4 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=119 time=10.2 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=119 time=7.62 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=119 time=9.29 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=119 time=11.8 ms
^Z
[1]+  Stopped                  ping -4 8.8.8.8
tamam@tamam-VirtualBox:~$

```

A successful ping to 8.8.8.8 confirms external connectivity before WAN detection tests.

4. Suricata Startup and Custom Rules

Figure 6: Suricata Configuration and Log Setup

```
tamam@tamam-VirtualBox:~$ sudo suricata -T -c /etc/suricata/suricata.yaml
[sudo] password for tamam:
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
i: suricata: Configuration provided was successfully loaded. Exiting.
tamam@tamam-VirtualBox:~$ ls /var/log/suricata-wan
eve.json fast.log stats.log suricata.log
tamam@tamam-VirtualBox:~$ sudo chmod 777 /var/log/suricata-wan
tamam@tamam-VirtualBox:~$ sudo suricata -c /etc/suricata/suricata.yaml -i wlx9cefd5f75df7 -l /var/log/suricata-wan
```

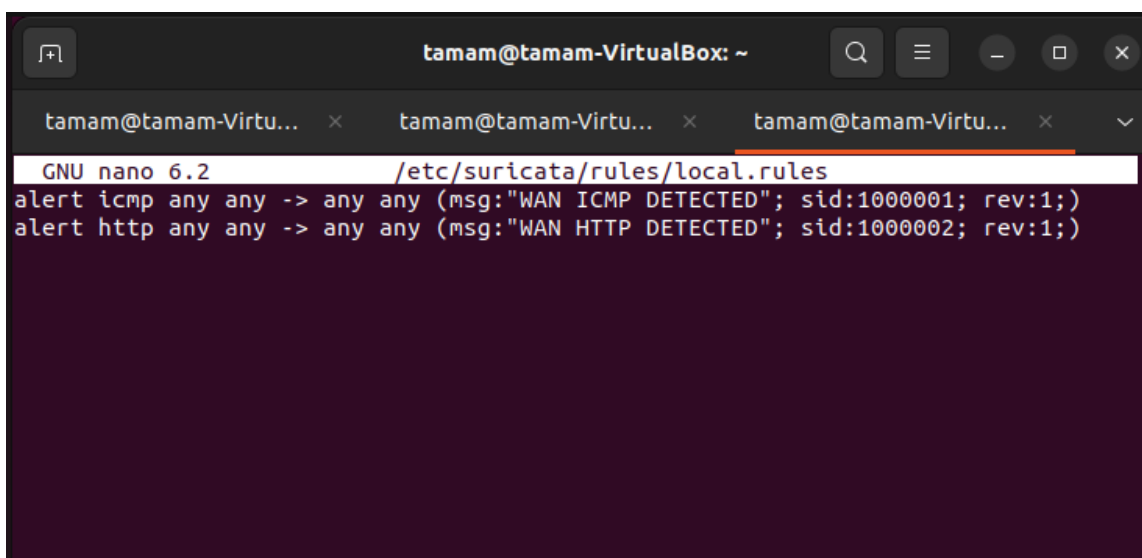
The Suricata configuration is validated successfully, and WAN log directories are prepared to store alerts separately.

Figure 7: Starting Suricata on WAN

```
2ec291b8cd .
tamam@tamam-VirtualBox:~$ sudo suricata -c /etc/suricata/suricata.yaml -i wlx9cefd5f75df7 -l /var/log/suricata-wan
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
i: threads: Threads created -> W: 2 FM: 1 FR: 1 Engine started.
```

Suricata is started on wlx9cefd5f75df7, with threads created and the engine running successfully.

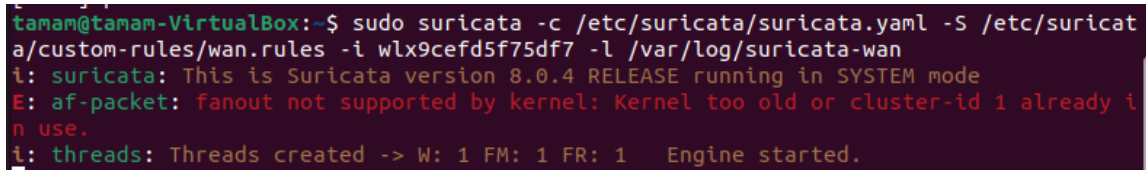
Figure 8: ICMP and HTTP Detection Rules



```
GNU nano 6.2 /etc/suricata/rules/local.rules
alert icmp any any -> any any (msg:"WAN ICMP DETECTED"; sid:1000001; rev:1;)
alert http any any -> any any (msg:"WAN HTTP DETECTED"; sid:1000002; rev:1;)
```

The local rules file includes simple ICMP and HTTP rules used to test Suricata alert generation.

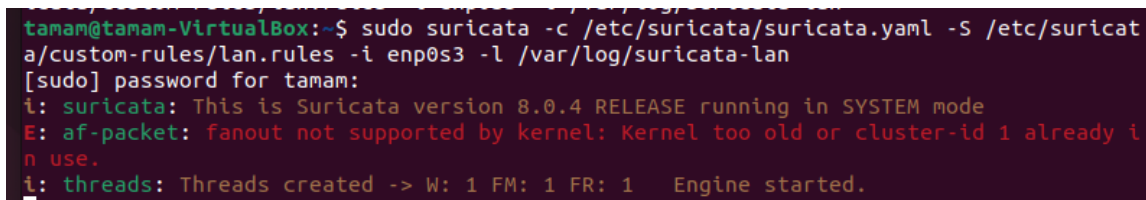
Figure 9: Running WAN Rules



```
tamam@tamam-VirtualBox:~$ sudo suricata -c /etc/suricata/suricata.yaml -S /etc/suricata/custom-rules/wan.rules -i wlx9cefd5f75df7 -l /var/log/suricata-wan
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
E: af-packet: fanout not supported by kernel: Kernel too old or cluster-id 1 already in use.
i: threads: Threads created -> W: 1 FM: 1 FR: 1 Engine started.
```

Suricata is launched with wan.rules on the WAN interface to enable external ICMP and HTTP detection.

Figure 10: Running LAN Rules



```
tamam@tamam-VirtualBox:~$ sudo suricata -c /etc/suricata/suricata.yaml -S /etc/suricata/custom-rules/lan.rules -i enp0s3 -l /var/log/suricata-lan
[sudo] password for tamam:
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
E: af-packet: fanout not supported by kernel: Kernel too old or cluster-id 1 already in use.
i: threads: Threads created -> W: 1 FM: 1 FR: 1 Engine started.
```

Suricata is launched with lan.rules on the internal enp0s3 interface to monitor local traffic.

Figure 11: WAN HTTP Alerts

```

tamam@tamam-VirtualBox: ~
inet 192.168.0.157/24 brd 192.168.0.255 scope global dynamic noprefixroute wlx9cef
d5f75df7
    valid_lft 83901sec preferred_lft 83901sec
inet6 fe80::4b1b:c7bc:e828:28c1/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
tamam@tamam-VirtualBox:~$ sudo tail -f /var/log/suricata-wan/fast.log
[sudo] password for tamam:
05/16/2026-10:37:11.467046  [**] [1:1000201:1] WAN ICMP DETECTED [**] [Classification:
(null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:1c4e:1a2e:657c:5f6f:133 -> ff02
:0000:0000:0000:0000:0000:0000:0000:0002:0
05/16/2026-10:38:27.473763  [**] [1:1000201:1] WAN ICMP DETECTED [**] [Classification:
(null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:1c4e:1a2e:657c:5f6f:133 -> ff02
:0000:0000:0000:0000:0000:0000:0000:0002:0
05/16/2026-10:41:47.098422  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 185.125.190.101:80 -> 192.168.0.157:59438
05/16/2026-10:41:47.098467  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80
05/16/2026-10:41:47.098641  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80
05/16/2026-10:41:47.100496  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 185.125.190.101:80 -> 192.168.0.157:59438
05/16/2026-10:41:47.100527  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80
05/16/2026-10:41:47.100497  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 185.125.190.101:80 -> 192.168.0.157:59438
05/16/2026-10:41:47.686009  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80
05/16/2026-10:41:48.260985  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80
05/16/2026-10:41:49.412959  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80
05/16/2026-10:41:51.653903  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80
05/16/2026-10:41:56.389029  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80
05/16/2026-10:42:06.292926  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80
05/16/2026-10:42:24.036939  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80
05/16/2026-10:43:02.436955  [**] [1:1000202:1] WAN HTTP DETECTED [**] [Classification:
(null)] [Priority: 3] {TCP} 192.168.0.157:59438 -> 185.125.190.101:80

```

The fast.log file shows repeated WAN HTTP alerts, confirming that the HTTP detection rule is working.

Figure 12: Generating ICMP from Kali

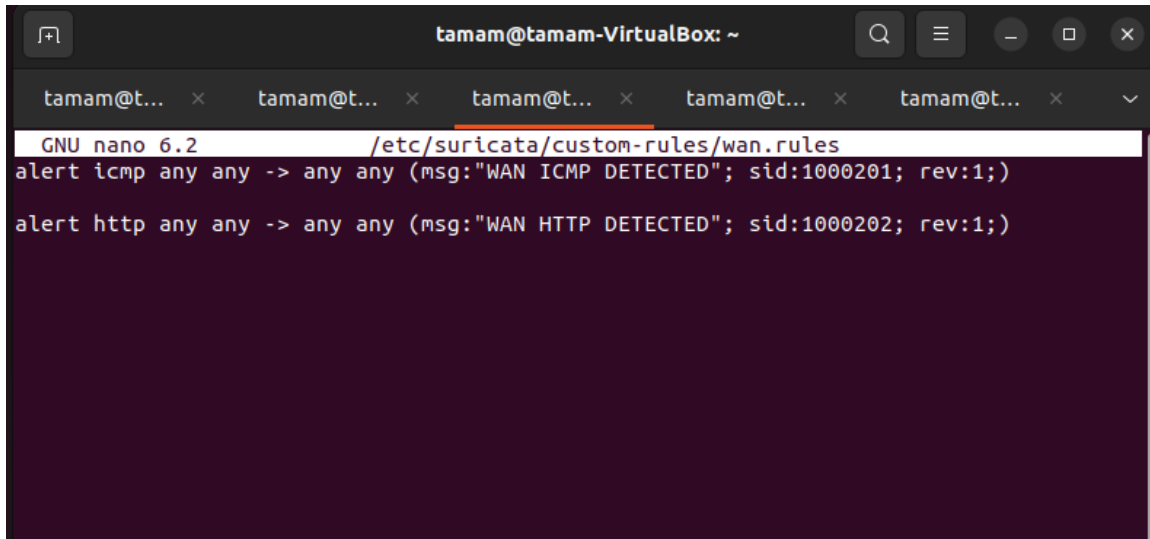
```
(kali㉿kali)-[~]  
$ ping 10.0.2.15  
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.  
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.928 ms  
64 bytes from 10.0.2.15: icmp_seq=2 ttl=64 time=0.979 ms  
64 bytes from 10.0.2.15: icmp_seq=3 ttl=64 time=0.957 ms  
64 bytes from 10.0.2.15: icmp_seq=4 ttl=64 time=2.49 ms  
64 bytes from 10.0.2.15: icmp_seq=5 ttl=64 time=1.28 ms  
64 bytes from 10.0.2.15: icmp_seq=6 ttl=64 time=0.872 ms  
64 bytes from 10.0.2.15: icmp_seq=7 ttl=64 time=1.34 ms  
64 bytes from 10.0.2.15: icmp_seq=8 ttl=64 time=1.31 ms  
64 bytes from 10.0.2.15: icmp_seq=9 ttl=64 time=2.27 ms  
64 bytes from 10.0.2.15: icmp_seq=10 ttl=64 time=1.41 ms  
64 bytes from 10.0.2.15: icmp_seq=11 ttl=64 time=0.972 ms  
64 bytes from 10.0.2.15: icmp_seq=12 ttl=64 time=1.08 ms  
^Z  
zsh: suspended ping 10.0.2.15  
  
(kali㉿kali)-[~]  
$
```

Kali sends ping traffic to 10.0.2.15 successfully, generating ICMP traffic for testing.

Figure 13: LAN ICMP Rule

```
tamam@tamam-VirtualBox: ~  
GNU nano 6.2 /etc/suricata/custom-rules/lan.rules  
alert icmp any any -> any any (msg:"LAN ICMP DETECTED"; sid:1000101; rev:1;)
```

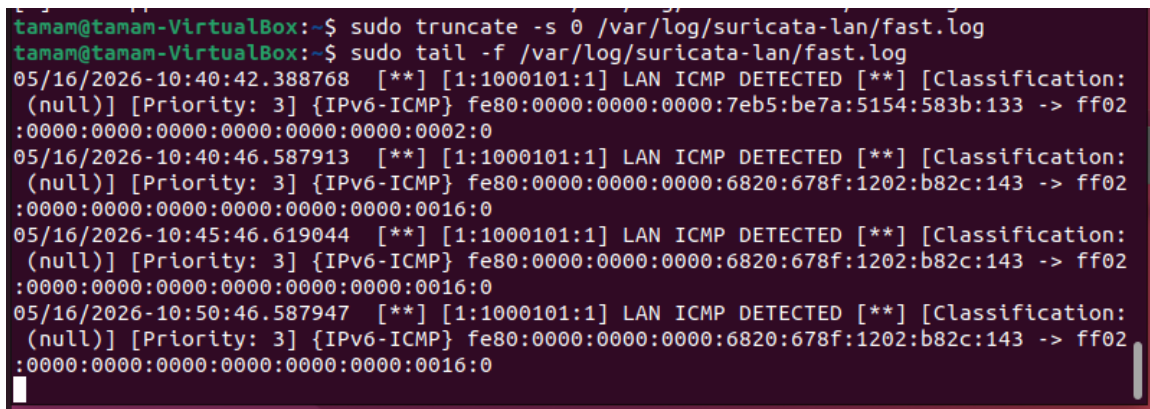
The lan.rules file contains an ICMP detection rule that generates LAN ICMP DETECTED alerts.

Figure 14: Basic WAN Rules


```
tamam@tamam-VirtualBox: ~
GNU nano 6.2 /etc/suricata/custom-rules/wan.rules
alert icmp any any -> any any (msg:"WAN ICMP DETECTED"; sid:1000201; rev:1;)

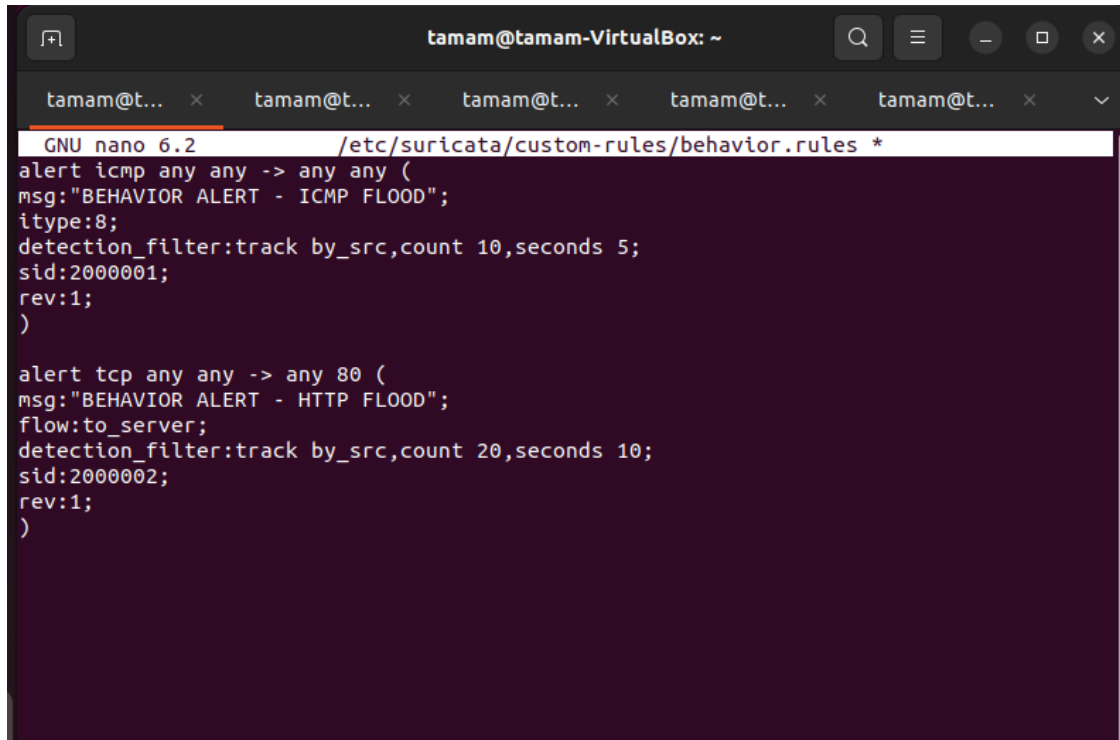
alert http any any -> any any (msg:"WAN HTTP DETECTED"; sid:1000202; rev:1;)
```

The wan.rules file contains custom ICMP and HTTP detection rules for external traffic monitoring.

Figure 15: LAN ICMP Detection Results


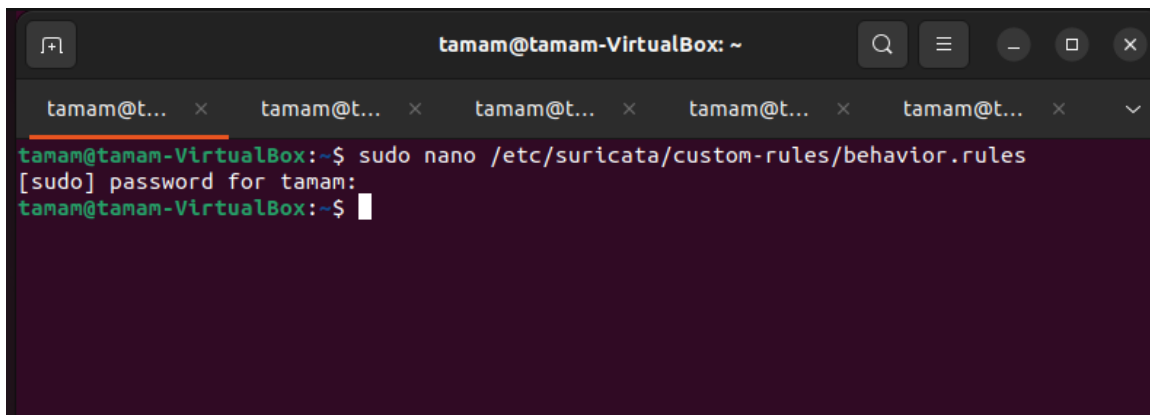
```
tamam@tamam-VirtualBox:~$ sudo truncate -s 0 /var/log/suricata-lan/fast.log
tamam@tamam-VirtualBox:~$ sudo tail -f /var/log/suricata-lan/fast.log
05/16/2026-10:40:42.388768  [**] [1:1000101:1] LAN ICMP DETECTED [**] [Classification:
(null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:7eb5:be7a:5154:583b:133 -> ff02
:0000:0000:0000:0000:0000:0000:0000:0002:0
05/16/2026-10:40:46.587913  [**] [1:1000101:1] LAN ICMP DETECTED [**] [Classification:
(null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:6820:678f:1202:b82c:143 -> ff02
:0000:0000:0000:0000:0000:0000:0000:0016:0
05/16/2026-10:45:46.619044  [**] [1:1000101:1] LAN ICMP DETECTED [**] [Classification:
(null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:6820:678f:1202:b82c:143 -> ff02
:0000:0000:0000:0000:0000:0000:0000:0016:0
05/16/2026-10:50:46.587947  [**] [1:1000101:1] LAN ICMP DETECTED [**] [Classification:
(null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:6820:678f:1202:b82c:143 -> ff02
:0000:0000:0000:0000:0000:0000:0000:0016:0
```

The LAN log shows multiple ICMP alerts, confirming that Suricata monitors the internal interface and generates alerts.

Figure 16: Behavior Detection Rules

```
tamam@tamam-VirtualBox: ~  
GNU nano 6.2 /etc/suricata/custom-rules/behavior.rules *  
alert icmp any any -> any any (  
  msg:"BEHAVIOR ALERT - ICMP FLOOD";  
  itype:8;  
  detection_filter:track by_src,count 10,seconds 5;  
  sid:2000001;  
  rev:1;  
)  
  
alert tcp any any -> any 80 (  
  msg:"BEHAVIOR ALERT - HTTP FLOOD";  
  flow:to_server;  
  detection_filter:track by_src,count 20,seconds 10;  
  sid:2000002;  
  rev:1;  
)
```

behavior.rules defines ICMP Flood and HTTP Flood rules using detection_filter to detect repeated traffic in a short time.

Figure 17: Opening behavior.rules

```
tamam@tamam-VirtualBox: ~  
tamam@tamam-VirtualBox:~$ sudo nano /etc/suricata/custom-rules/behavior.rules  
[sudo] password for tamam:  
tamam@tamam-VirtualBox:~$
```

The behavior.rules file is opened to edit or review behavioral rules for repeated attack detection.

5. ICMP, HTTP Flood, and Scanning Tests

Figure 18: ICMP Flood Detection

The figure consists of two side-by-side terminal windows. The left window is titled 'kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VM VirtualBox' and shows a Kali Linux terminal session. The user is running a loop of ping commands to 10.0.2.15, with the output showing successful connections and ping statistics. The right window is titled 'Monitoring Server [Running] - Oracle VM VirtualBox' and shows a terminal session with Suricata logs. The logs display multiple 'BEHAVIOR ALERT - ICMP FLOOD' messages, indicating that the system has detected a flood of ICMP traffic.

```

kali@kali:~$ for i in {1..50}; do ping -c 1 10.0.2.15; done
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data:
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.32 ms

--- 10.0.2.15 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/ndev = 1.324/1.324/1.324/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data:
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.20 ms

--- 10.0.2.15 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/ndev = 1.199/1.199/1.199/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data:
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.829 ms

--- 10.0.2.15 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/ndev = 0.829/0.829/0.829/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data:
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.687 ms

--- 10.0.2.15 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/ndev = 0.687/0.687/0.687/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data:
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.880 ms

--- 10.0.2.15 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/ndev = 0.880/0.880/0.880/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data:
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.53 ms

--- 10.0.2.15 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/ndev = 1.531/1.531/1.531/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data:
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.924 ms

--- 10.0.2.15 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/ndev = 0.924/0.924/0.924/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data:
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.05 ms

--- 10.0.2.15 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/ndev = 1.047/1.047/1.047/0.000 ms

```

```

tamam@tamam-VirtualBox:~$ sudo tail -f /var/log/suricata-behavior/fast.log
05/16/2026-11:35:33.098115 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.102781 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.108521 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.113336 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.117667 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.122720 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.127277 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.132756 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.137985 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.143582 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.149341 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.154249 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.159777 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.164372 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.168777 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.174669 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.179185 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.185234 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.191872 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.197675 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.202336 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] [ICMP] 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.208503 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:

```

Kali generates repeated ping traffic, while Suricata logs BEHAVIOR ALERT - ICMP FLOOD alerts.

Figure 19: Repeated Ping Test

```

(kali@kali)-[~]
$ for i in {1..50}; do ping -c 1 10.0.2.15; done
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.32 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.324/1.324/1.324/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.20 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.199/1.199/1.199/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.829 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.829/0.829/0.829/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.687 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.687/0.687/0.687/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.880 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.880/0.880/0.880/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.53 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.531/1.531/1.531/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.924 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.924/0.924/0.924/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.05 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.047/1.047/1.047/0.000 ms

```

Multiple ping requests are sent from Kali to create a behavior pattern that can be detected as a flood.

Figure 20: Continued ICMP Test

```
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.657 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.657/0.657/0.657/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.800 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.800/0.800/0.800/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.17 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.170/1.170/1.170/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.802 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.802/0.802/0.802/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.869 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.869/0.869/0.869/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.649 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.649/0.649/0.649/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.736 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.736/0.736/0.736/0.000 ms
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.937 ms

— 10.0.2.15 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.937/0.937/0.937/0.000 ms

—(kali@kali)-[~]
```

The repeated ping test continues with no packet loss, confirming successful traffic generation.

Figure 21: HTTP Flood Alerts

```

tamam@tamam-VirtualBox: ~
tamam@tam... x  tamam@tam... x  tamam@tam... x  tamam@tam... x  tamam@tam... x
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.197675  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.202336  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.208503  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.213622  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.218626  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.224040  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.228675  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.233780  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.240589  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.245493  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.250153  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.254281  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.259479  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.263619  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.268854  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.274761  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.282758  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.289636  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.293934  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.300343  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-11:35:33.306091  [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD [**] [Classification:
(null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0

```

The log shows many HTTP Flood alerts caused by repeated requests to the server.

Figure 22: Behavior Rules Loading Test

```
tamam@tamam-VirtualBox:~$ sudo nano /etc/suricata/custom-rules/behavior.rules
[sudo] password for tamam:
tamam@tamam-VirtualBox:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -S /etc/suricata/custom-rules/behavior.rules
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
i: suricata: Configuration provided was successfully loaded. Exiting.
tamam@tamam-VirtualBox:~$
```

Suricata is tested or started with behavior.rules to confirm that behavioral rules are loaded correctly.

Figure 23: Editing HTTP Flood Rule

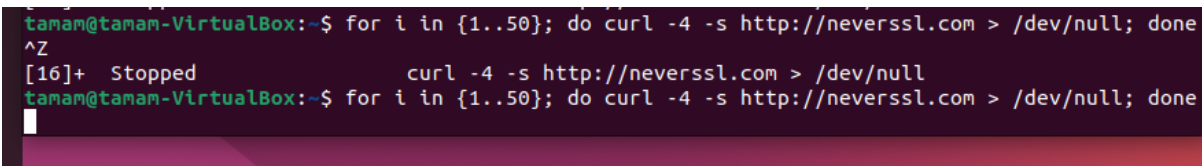
```
tamam@tamam-VirtualBox:~$ nano /etc/suricata/custom-rules/behavior.rules
GNU nano 6.2 /etc/suricata/custom-rules/behavior.rules
alert icmp any any -> any any (msg:"BEHAVIOR ALERT - ICMP FLOOD"; itype:8; detection_filter:tra
alert tcp any any -> any 80 (msg:"BEHAVIOR ALERT - HTTP FLOOD"; flow:to_server; detection_filte
```

The behavior.rules file is shown while adding or editing the HTTP Flood rule for port 80.

Figure 24: Starting Suricata with Behavior Rules

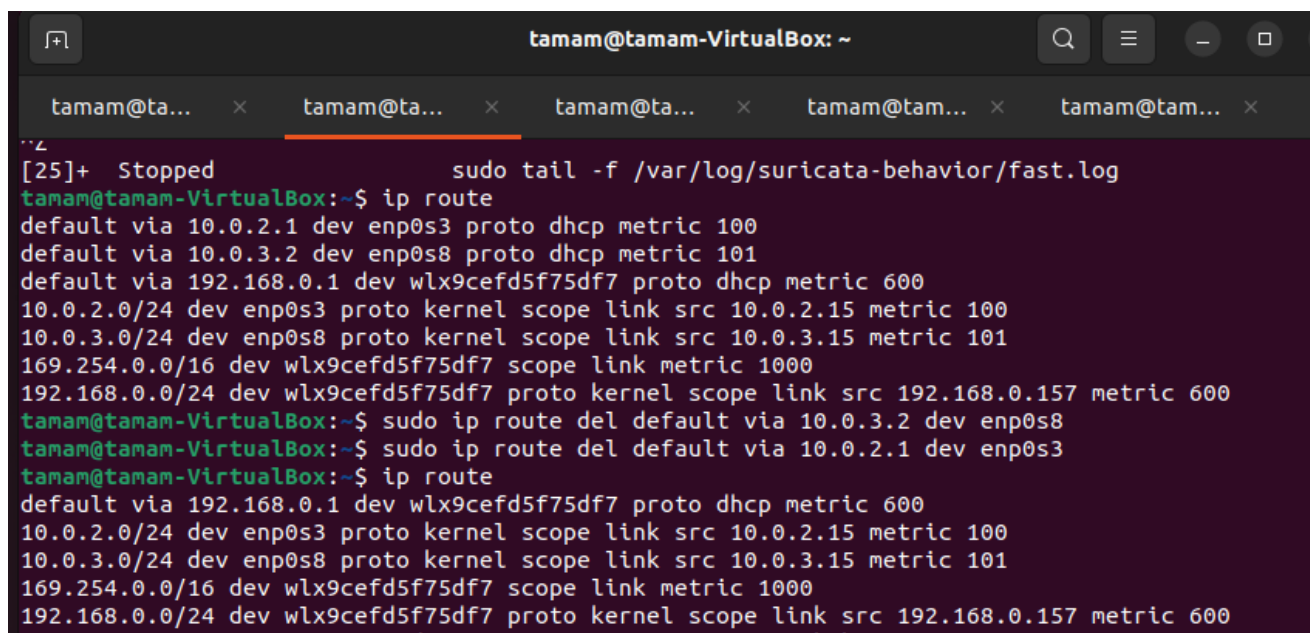
```
tamam@tamam-VirtualBox:~$ sudo pkill suricata
tamam@tamam-VirtualBox:~$ sudo mkdir -p /var/log/suricata-behavior
sudo chmod 777 /var/log/suricata-behavior
sudo suricata -c /etc/suricata/suricata.yaml -S /etc/suricata/custom-rules/behavior.rules -i enp0s3 -l /var/log/suricata-behavior
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
E: af-packet: fanout not supported by kernel: Kernel too old or cluster-id 1 already in use.
i: threads: Threads created -> W: 1 FM: 1 FR: 1 Engine started.
```

Suricata is started with behavior.rules on the internal interface to monitor behavioral attacks.

Figure 25: Generating HTTP Requests with curl


```
tamam@tamam-VirtualBox:~$ for i in {1..50}; do curl -4 -s http://neverssl.com > /dev/null; done
^Z
[16]+  Stopped                  curl -4 -s http://neverssl.com > /dev/null
tamam@tamam-VirtualBox:~$ for i in {1..50}; do curl -4 -s http://neverssl.com > /dev/null; done
```

A curl loop sends repeated HTTP requests to the server to test HTTP Flood detection.

Figure 26: WAN Route Confirmation


```
tamam@tamam-VirtualBox: ~
tamam@ta... x tamam@ta... x tamam@ta... x tamam@tam... x tamam@tam... x
[25]+  Stopped                  sudo tail -f /var/log/suricata-behavior/fast.log
tamam@tamam-VirtualBox:~$ ip route
default via 10.0.2.1 dev enp0s3 proto dhcp metric 100
default via 10.0.3.2 dev enp0s8 proto dhcp metric 101
default via 192.168.0.1 dev wlx9cefd5f75df7 proto dhcp metric 600
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.3.0/24 dev enp0s8 proto kernel scope link src 10.0.3.15 metric 101
169.254.0.0/16 dev wlx9cefd5f75df7 scope link metric 1000
192.168.0.0/24 dev wlx9cefd5f75df7 proto kernel scope link src 192.168.0.157 metric 600
tamam@tamam-VirtualBox:~$ sudo ip route del default via 10.0.3.2 dev enp0s8
tamam@tamam-VirtualBox:~$ sudo ip route del default via 10.0.2.1 dev enp0s3
tamam@tamam-VirtualBox:~$ ip route
default via 192.168.0.1 dev wlx9cefd5f75df7 proto dhcp metric 600
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.3.0/24 dev enp0s8 proto kernel scope link src 10.0.3.15 metric 101
169.254.0.0/16 dev wlx9cefd5f75df7 scope link metric 1000
192.168.0.0/24 dev wlx9cefd5f75df7 proto kernel scope link src 192.168.0.157 metric 600
```

The routing table is adjusted to confirm that external connectivity passes through the wireless interface.

Figure 27: Repeated Behavioral Alerts

```

tamam@tamam-VirtualBox:~$ sudo tail -f /var/log/suricata-behavior/fast.log
05/16/2026-13:13:42.816941  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:50242 -> 142.250.75.67:80
05/16/2026-13:13:43.525403  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:50596 -> 34.223.124.45:80
05/16/2026-13:13:44.285439  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:50242 -> 142.250.75.67:80
05/16/2026-13:13:44.285578  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:55582 -> 23.11.40.157:80
05/16/2026-13:13:44.286064  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:55570 -> 23.11.40.157:80
05/16/2026-13:13:44.287956  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:58374 -> 34.107.221.82:80
05/16/2026-13:13:44.287971  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:58362 -> 34.107.221.82:80
05/16/2026-13:13:44.298850  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:50242 -> 142.250.75.67:80
05/16/2026-13:13:44.304993  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:58374 -> 34.107.221.82:80
05/16/2026-13:13:44.305013  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:58362 -> 34.107.221.82:80
05/16/2026-13:13:44.675089  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:55570 -> 23.11.40.157:80
05/16/2026-13:13:44.694168  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:55582 -> 23.11.40.157:80
05/16/2026-13:13:45.062359  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:55570 -> 23.11.40.157:80
05/16/2026-13:13:45.109970  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:55582 -> 23.11.40.157:80
05/16/2026-13:13:45.829598  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:55570 -> 23.11.40.157:80
05/16/2026-13:13:45.957129  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:55582 -> 23.11.40.157:80
05/16/2026-13:13:46.341156  [**] [1:2000002:3] BEHAVIOR ALERT - HTTP FLOOD [**] [Classification
: (null)] [Priority: 3] {TCP} 192.168.0.157:50596 -> 34.223.124.45:80

```

The Suricata log shows repeated behavioral alerts caused by repeated connections or packets.

Figure 28: HTTP Flood Results

Hybrid Multi-Interface IDS/IPS Platform Using Suricata

```
tamam@tamam-VirtualBox: ~  
tamam... x tamam... x tamam... x tamam... x tamam... x tamam@... x  
192.168.0.0/24 dev wlx9cefd5f75df7 proto kernel scope link src 192.168.0.157 metric 600  
tamam@tamam-VirtualBox: ~$ sudo tail -f /var/log/suricata-behavior/fast.log  
05/16/2026-13:43:52.936584 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:53.038625 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:53.139926 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:53.242760 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:53.343655 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:53.444395 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:53.547348 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:53.648328 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:53.749980 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:53.850510 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:53.953488 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:54.055555 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:54.156409 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:54.259633 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:54.362173 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:54.464009 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:54.565769 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:54.666766 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas  
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0  
05/16/2026-13:43:54.767145 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas
```

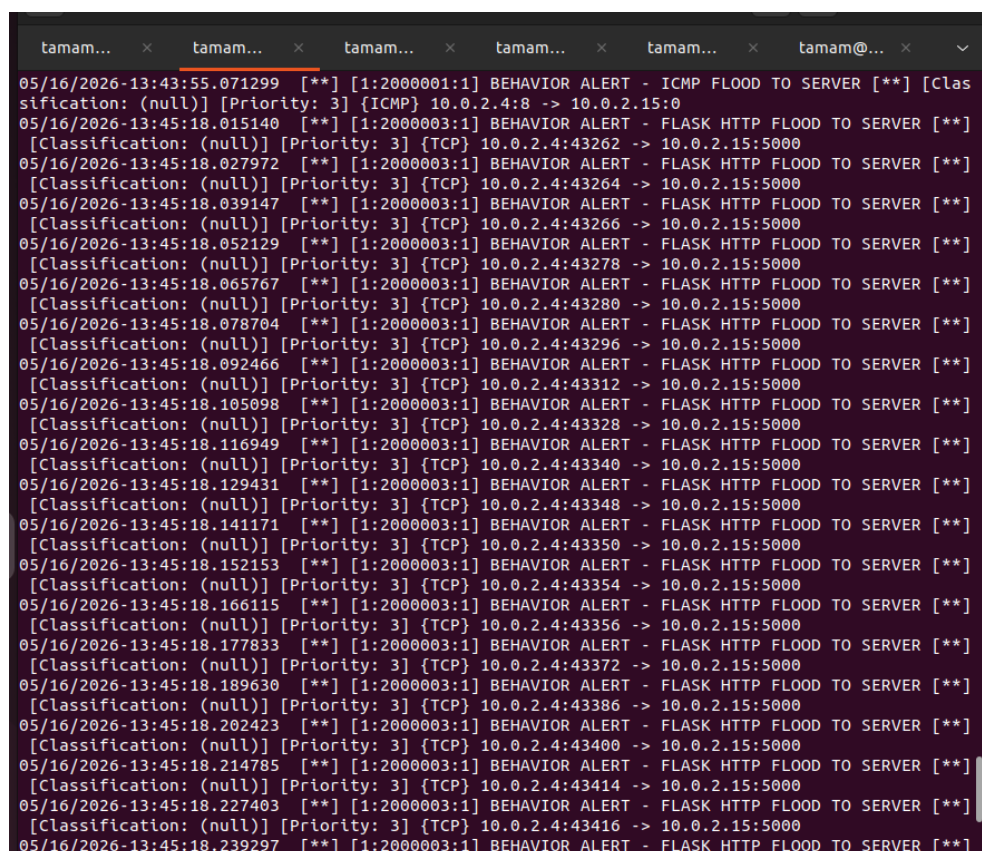
HTTP Flood alerts appear after repeated requests are generated, confirming successful detection.

Figure 29: Kali Ping Loop

```
(kali@kali)-[~]  
$ ping -i 0.1 10.0.2.15  
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.  
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.740 ms  
64 bytes from 10.0.2.15: icmp_seq=2 ttl=64 time=1.70 ms  
64 bytes from 10.0.2.15: icmp_seq=3 ttl=64 time=1.48 ms  
64 bytes from 10.0.2.15: icmp_seq=4 ttl=64 time=2.19 ms  
64 bytes from 10.0.2.15: icmp_seq=5 ttl=64 time=0.891 ms  
64 bytes from 10.0.2.15: icmp_seq=6 ttl=64 time=1.07 ms  
64 bytes from 10.0.2.15: icmp_seq=7 ttl=64 time=0.824 ms  
64 bytes from 10.0.2.15: icmp_seq=8 ttl=64 time=1.37 ms  
64 bytes from 10.0.2.15: icmp_seq=9 ttl=64 time=1.34 ms  
64 bytes from 10.0.2.15: icmp_seq=10 ttl=64 time=0.895 ms  
64 bytes from 10.0.2.15: icmp_seq=11 ttl=64 time=0.951 ms  
64 bytes from 10.0.2.15: icmp_seq=12 ttl=64 time=0.876 ms  
64 bytes from 10.0.2.15: icmp_seq=13 ttl=64 time=1.25 ms  
64 bytes from 10.0.2.15: icmp_seq=14 ttl=64 time=1.24 ms  
64 bytes from 10.0.2.15: icmp_seq=15 ttl=64 time=0.888 ms  
64 bytes from 10.0.2.15: icmp_seq=16 ttl=64 time=1.23 ms  
64 bytes from 10.0.2.15: icmp_seq=17 ttl=64 time=1.66 ms  
64 bytes from 10.0.2.15: icmp_seq=18 ttl=64 time=0.969 ms  
64 bytes from 10.0.2.15: icmp_seq=19 ttl=64 time=1.08 ms  
64 bytes from 10.0.2.15: icmp_seq=20 ttl=64 time=0.872 ms  
64 bytes from 10.0.2.15: icmp_seq=21 ttl=64 time=2.28 ms  
64 bytes from 10.0.2.15: icmp_seq=22 ttl=64 time=1.98 ms  
64 bytes from 10.0.2.15: icmp_seq=23 ttl=64 time=1.18 ms  
64 bytes from 10.0.2.15: icmp_seq=24 ttl=64 time=0.952 ms  
64 bytes from 10.0.2.15: icmp_seq=25 ttl=64 time=1.32 ms  
64 bytes from 10.0.2.15: icmp_seq=26 ttl=64 time=1.61 ms  
64 bytes from 10.0.2.15: icmp_seq=27 ttl=64 time=1.35 ms  
64 bytes from 10.0.2.15: icmp_seq=28 ttl=64 time=1.89 ms  
64 bytes from 10.0.2.15: icmp_seq=29 ttl=64 time=0.937 ms  
64 bytes from 10.0.2.15: icmp_seq=30 ttl=64 time=0.923 ms  
64 bytes from 10.0.2.15: icmp_seq=31 ttl=64 time=1.71 ms  
64 bytes from 10.0.2.15: icmp_seq=32 ttl=64 time=1.57 ms  
^Z  
zsh: suspended ping -i 0.1 10.0.2.15  
(kali@kali)-[~]
```

Kali sends repeated ping packets to the monitoring server to test repeated traffic detection.

Figure 30: ICMP Flood Log



```
tamam... x tamam... x tamam... x tamam... x tamam... x tamam@... x v
05/16/2026-13:43:55.071299 [**] [1:2000001:1] BEHAVIOR ALERT - ICMP FLOOD TO SERVER [**] [Clas
sification: (null)] [Priority: 3] {ICMP} 10.0.2.4:8 -> 10.0.2.15:0
05/16/2026-13:45:18.015140 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43262 -> 10.0.2.15:5000
05/16/2026-13:45:18.027972 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43264 -> 10.0.2.15:5000
05/16/2026-13:45:18.039147 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43266 -> 10.0.2.15:5000
05/16/2026-13:45:18.052129 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43278 -> 10.0.2.15:5000
05/16/2026-13:45:18.065767 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43280 -> 10.0.2.15:5000
05/16/2026-13:45:18.078704 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43296 -> 10.0.2.15:5000
05/16/2026-13:45:18.092466 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43312 -> 10.0.2.15:5000
05/16/2026-13:45:18.105098 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43328 -> 10.0.2.15:5000
05/16/2026-13:45:18.116949 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43340 -> 10.0.2.15:5000
05/16/2026-13:45:18.129431 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43348 -> 10.0.2.15:5000
05/16/2026-13:45:18.141171 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43350 -> 10.0.2.15:5000
05/16/2026-13:45:18.152153 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43354 -> 10.0.2.15:5000
05/16/2026-13:45:18.166115 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43356 -> 10.0.2.15:5000
05/16/2026-13:45:18.177833 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43372 -> 10.0.2.15:5000
05/16/2026-13:45:18.189630 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43386 -> 10.0.2.15:5000
05/16/2026-13:45:18.202423 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43400 -> 10.0.2.15:5000
05/16/2026-13:45:18.214785 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43414 -> 10.0.2.15:5000
05/16/2026-13:45:18.227403 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43416 -> 10.0.2.15:5000
05/16/2026-13:45:18.239297 [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
```

fast.log displays many ICMP Flood alerts generated by repeated ping traffic.

Figure 31: Generating POST Requests



```
Left: Suspended ping 1 0.1 10.0.2.15
(kali@kali)-[~]
$ for i in {1..30}; do curl -s -X POST http://10.0.2.15:5000/subscribe > /dev/null; done
(kali@kali)-[~]
```

Repeated POST requests are sent to the web service to test application-layer flood detection.

Figure 32: Flash HTTP Flood Alerts

```

[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43420 -> 10.0.2.15:5000
05/16/2026-13:45:18.251514  [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 10.0.2.4:43432 -> 10.0.2.15:5000
05/16/2026-14:10:19.324710  [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.197:49266 -> 192.168.0.157:5000
05/16/2026-14:10:28.368153  [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.197:49288 -> 192.168.0.157:5000
05/16/2026-14:10:29.148647  [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.197:49290 -> 192.168.0.157:5000
05/16/2026-14:10:29.962408  [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.197:49292 -> 192.168.0.157:5000
05/16/2026-14:12:09.752671  [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.197:49336 -> 192.168.0.157:5000
05/16/2026-14:12:10.799047  [**] [1:2000003:1] BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER [**]
[Classification: (null)] [Priority: 3] {TCP} 192.168.0.197:49338 -> 192.168.0.157:5000

```

The log displays alerts related to Flash HTTP Flood behavior directed at the server.

Figure 33: Testing Flash HTTP Flood Rule

```

[Suricata Stopped]
tamam@tamam-VirtualBox:~$ sudo suricata -c /etc/suricata/suricata.yaml -S /etc/suricata/custom-
rules/behavior.rules -i wlx9cefd5f75df7 -l /var/log/suricata-behavior
[sudo] password for tamam:
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
W: detect: rule 2000003: SYN-only to port(s) 5000:5000 w/o direction specified, disabling for t
oclient direction
W: detect: rule 2000004: SYN-only to port(s) 80:80 w/o direction specified, disabling for tocli
ent direction
E: af-packet: fanout not supported by kernel: Kernel too old or cluster-id 1 already in use.
i: threads: Threads created -> W: 1 FM: 1 FR: 1 Engine started.

```

Suricata is tested with a Flash HTTP Flood rule, and a syntax issue appears that requires correction.

Figure 34: Retesting the Rule After Editing

```

tamam@tamam-VirtualBox:~$ sudo pkill -9 suricata
sudo suricata -c /etc/suricata/suricata.yaml -S /etc/suricata/custom-rules/behavior.rules -i en
p0s3 -l /var/log/suricata-behavior
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
W: detect: rule 2000003: SYN-only to port(s) 5000:5000 w/o direction specified, disabling for t
oclient direction
W: detect: rule 2000004: SYN-only to port(s) 80:80 w/o direction specified, disabling for tocli
ent direction
E: af-packet: fanout not supported by kernel: Kernel too old or cluster-id 1 already in use.
i: threads: Threads created -> W: 1 FM: 1 FR: 1 Engine started.

```

The rule is retested after editing to verify whether Suricata accepts it or still reports an error.

Figure 35: Nmap Scan

```

tamam@tamam-VirtualBox:~$ python3 /usr/local/bin/subscribe_server.py
* Serving Flask app 'subscribe_server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
on MSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.2.15:5000
Press CTRL+C to quit
192.168.0.197 - - [16/May/2026 14:01:36] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:01:36] "GET /favicon.ico HTTP/1.1" 404 -
192.168.0.197 - - [16/May/2026 14:09:58] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:09:59] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:00] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:01] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:03] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:04] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:05] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:05] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:07] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:08] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:09] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:10] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:10] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:11] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:12] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:13] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:13] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:15] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:16] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:18] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:19] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:21] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:21] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:22] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:23] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:23] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:24] "GET /subscribe HTTP/1.1" 405 -
192.168.0.197 - - [16/May/2026 14:10:25] "GET /subscribe HTTP/1.1" 405 -

```

An nmap scan is executed against the server to test detection of network scanning and open ports.

Figure 36: SYN, HTTP, and Nmap Rules

```

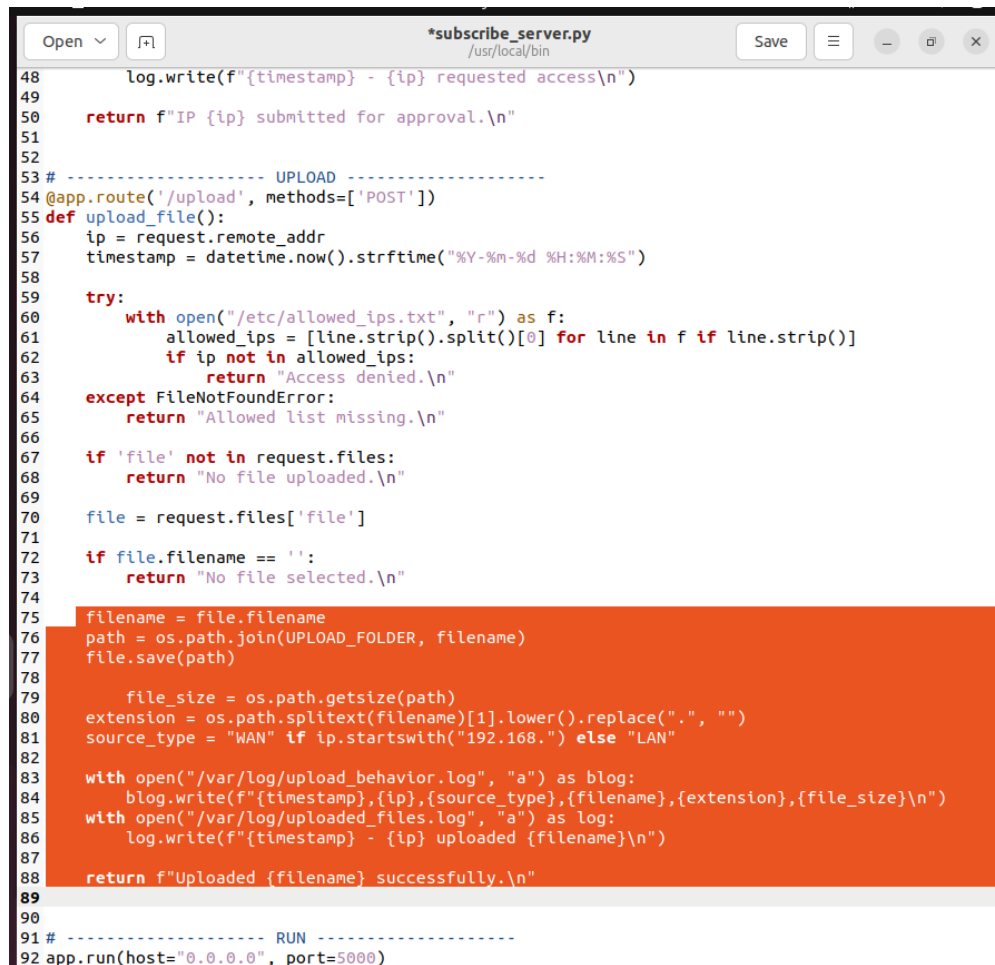
tamam@tamam-VirtualBox: ~
GNU nano 6.2 /etc/suricata/custom-rules/behavior.rules
alert icmp any any -> any any (msg:"BEHAVIOR ALERT - ICMP FLOOD TO SERVER"; itype:8; detection>
alert tcp any any -> any 5000 (msg:"BEHAVIOR ALERT - FLASK HTTP FLOOD TO SERVER"; flags:S; det>
alert tcp any any -> any [80,443] (msg:"BEHAVIOR ALERT - HTTP/HTTPS FLOOD TO SERVER"; flags:S;>
alert tcp any any -> any 5000 (msg:"BEHAVIOR ALERT - PHONE/WEB FLOOD TO FLASK"; flags:S; detec>

```

The rules file contains multiple rules for detecting SYN flood, HTTP flood, and nmap scanning attempts.

6. Behavior Analyzer and File Upload Detection

Figure 37: Behavior Analyzer Code



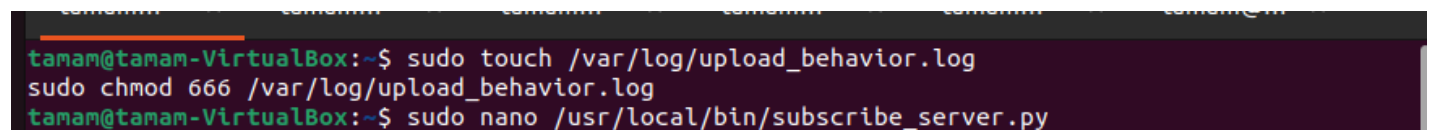
```

48     log.write(f"{timestamp} - {ip} requested access\n")
49
50     return f"IP {ip} submitted for approval.\n"
51
52
53 # ----- UPLOAD -----
54 @app.route('/upload', methods=['POST'])
55 def upload_file():
56     ip = request.remote_addr
57     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
58
59     try:
60         with open("/etc/allowed_ips.txt", "r") as f:
61             allowed_ips = [line.strip().split()[0] for line in f if line.strip()]
62             if ip not in allowed_ips:
63                 return "Access denied.\n"
64     except FileNotFoundError:
65         return "Allowed list missing.\n"
66
67     if 'file' not in request.files:
68         return "No file uploaded.\n"
69
70     file = request.files['file']
71
72     if file.filename == '':
73         return "No file selected.\n"
74
75     filename = file.filename
76     path = os.path.join(UPLOAD_FOLDER, filename)
77     file.save(path)
78
79     file_size = os.path.getsize(path)
80     extension = os.path.splitext(filename)[1].lower().replace(".", "")
81     source_type = "WAN" if ip.startswith("192.168.") else "LAN"
82
83     with open("/var/log/upload_behavior.log", "a") as blog:
84         blog.write(f"{timestamp},{ip},{source_type},{filename},{extension},{file_size}\n")
85     with open("/var/log/uploaded_files.log", "a") as log:
86         log.write(f"{timestamp} - {ip} uploaded {filename}\n")
87
88     return f"Uploaded {filename} successfully.\n"
89
90
91 # ----- RUN -----
92 app.run(host="0.0.0.0", port=5000)

```

Parts of the Python behavior analyzer script are shown, which analyzes logs and classifies events by threat type.

Figure 38: Preparing behavior.log



```

tamam@tamam-VirtualBox:~$ sudo touch /var/log/upload_behavior.log
sudo chmod 666 /var/log/upload_behavior.log
tamam@tamam-VirtualBox:~$ sudo nano /usr/local/bin/subscribe_server.py

```

The behavior log file is created or cleared and assigned proper permissions for the analyzer.

Figure 39: Behavior Analyzer Settings

```

GNU nano 6.2 /usr/local/bin/behavior_analyzer.py *
from collections import defaultdict
from time import sleep

LOG_FILE = "/var/log/upload_behavior.log"

dangerous_extensions = ["php", "exe", "bat", "sh"]

seen = set()
upload_counter = defaultdict(int)

print("[*] Behavioral Analyzer Started...")

while True:
    try:
        with open(LOG_FILE, "r") as f:
            lines = f.readlines()

        for line in lines:
            if line in seen:
                continue

            seen.add(line)

            try:
                timestamp, ip, source, filename, extension, size = line.strip().split(",")

                upload_counter[ip] += 1

                # -----
                # NORMAL USER
                # -----
                if extension in ["png", "jpg", "jpeg", "pdf"]:
                    print(f"[NORMAL] {ip} uploaded normal file: {filename}")

                # -----

            except:
                pass

    except:
        pass

    sleep(2)

```

The script shows reading and analysis settings, including file types or extensions to monitor.

Figure 40: Threat Detection Logic

```

GNU nano 6.2 /usr/local/bin/behavior_analyzer.py *

        upload_counter[ip] += 1

        # -----
        # NORMAL USER
        # -----
        if extension in ["png", "jpg", "jpeg", "pdf"]:
            print(f"[NORMAL] {ip} uploaded normal file: {filename}")

        # -----
        # SUSPICIOUS FILE
        # -----
        if extension in dangerous_extensions:
            print(f"[SUSPICIOUS] Dangerous file uploaded by {ip}: {filename}")

        # -----
        # EXCESSIVE UPLOADS
        # -----
        if upload_counter[ip] >= 5:
            print(f"[ALERT] Excessive uploads detected from {ip}")

        # -----
        # LARGE FILE
        # -----
        if int(size) > 5000000:
            print(f"[ALERT] Large file uploaded by {ip}")

    except:
        pass

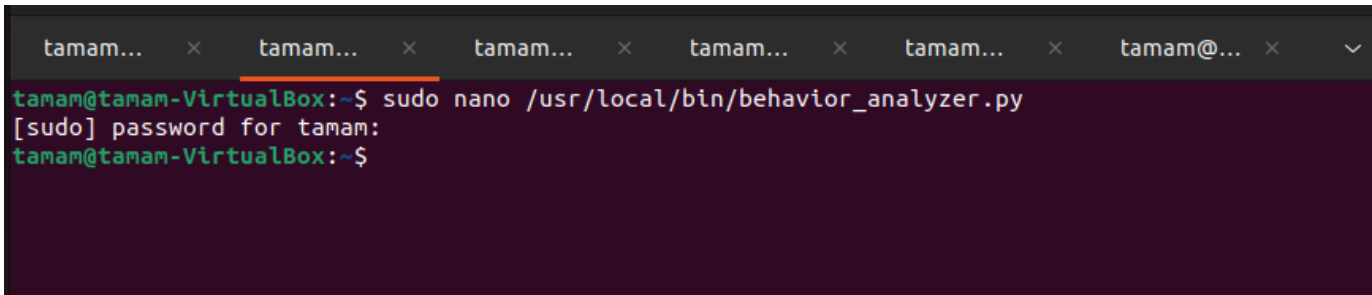
    except:
        pass

    sleep(2)

```

The script logic calculates repetition, size, and file type to determine threat severity.

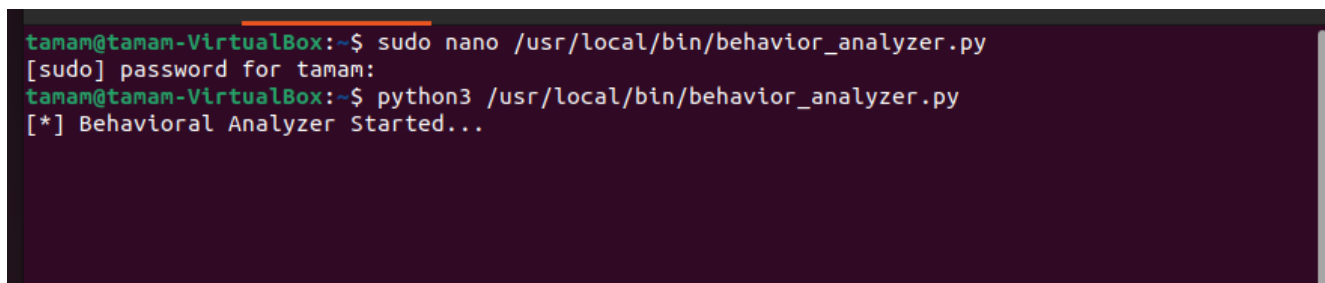
Figure 41: Clearing Global Behavior Log



```
tamam@tamam-VirtualBox:~$ sudo nano /usr/local/bin/behavior_analyzer.py
[sudo] password for tamam:
tamam@tamam-VirtualBox:~$
```

global_behavior.log is cleared before testing so that displayed results belong only to the current experiment.

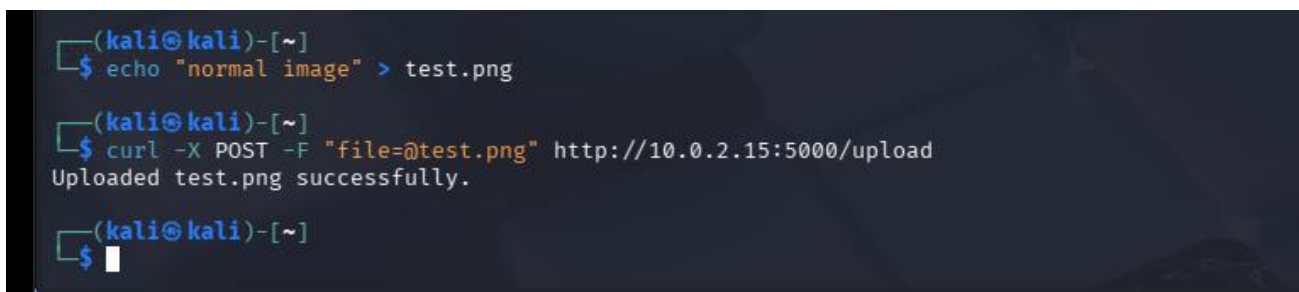
Figure 42: Running Behavior Analyzer



```
tamam@tamam-VirtualBox:~$ sudo nano /usr/local/bin/behavior_analyzer.py
[sudo] password for tamam:
tamam@tamam-VirtualBox:~$ python3 /usr/local/bin/behavior_analyzer.py
[*] Behavioral Analyzer Started...
```

behavior_analyzer.py is started and begins monitoring logs and analyzing new events.

Figure 43: Normal Upload: test.png

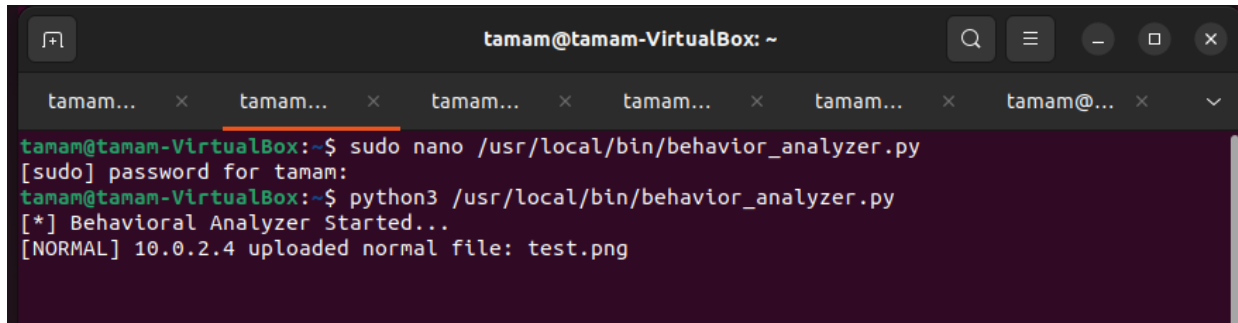


```
(kali@kali)-[~]
$ echo "normal image" > test.png

(kali@kali)-[~]
$ curl -X POST -F "file=@test.png" http://10.0.2.15:5000/upload
Uploaded test.png successfully.

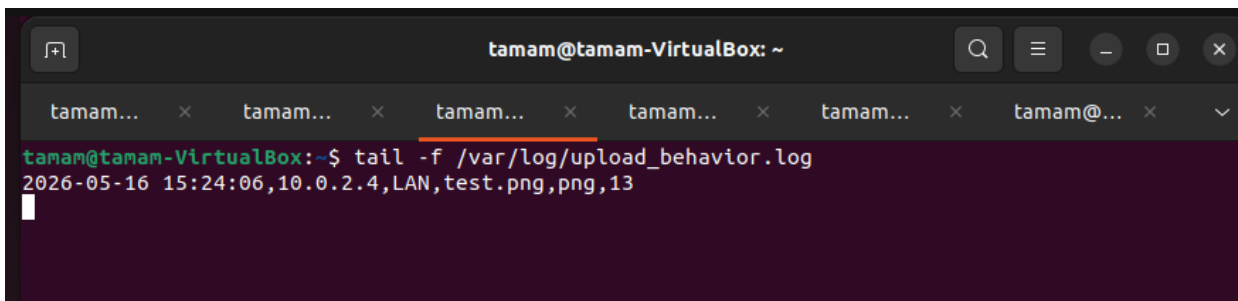
(kali@kali)-[~]
$
```

A normal image file is uploaded successfully to test the benign upload scenario.

Figure 44: Normal Upload Classification

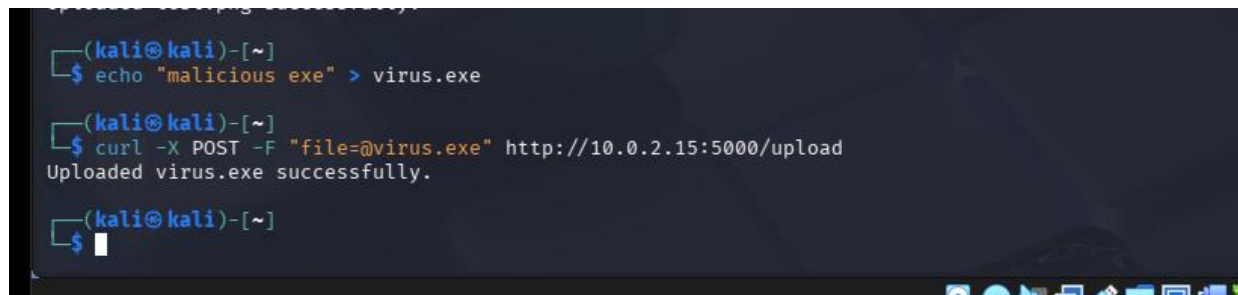
```
tamam@tamam-VirtualBox: ~  
tamam@tamam-VirtualBox:~$ sudo nano /usr/local/bin/behavior_analyzer.py  
[sudo] password for tamam:  
tamam@tamam-VirtualBox:~$ python3 /usr/local/bin/behavior_analyzer.py  
[*] Behavioral Analyzer Started...  
[NORMAL] 10.0.2.4 uploaded normal file: test.png
```

The analyzer classifies test.png as a normal upload without generating a high-risk alert.

Figure 45: Normal Upload Log

```
tamam@tamam-VirtualBox: ~  
tamam@tamam-VirtualBox:~$ tail -f /var/log/upload_behavior.log  
2026-05-16 15:24:06,10.0.2.4,LAN,test.png,png,13
```

global_behavior.log records the test.png upload as a behavioral event.

Figure 46: Executable Upload: virus.exe

```
(kali㉿kali)-[~]  
$ echo "malicious exe" > virus.exe  
  
(kali㉿kali)-[~]  
$ curl -X POST -F "file=@virus.exe" http://10.0.2.15:5000/upload  
Uploaded virus.exe successfully.  
  
(kali㉿kali)-[~]  
$
```

An exe file is uploaded to test the system's ability to detect dangerous extensions.

Figure 47: Suspicious Executable Detection

```
tamam@tamam-VirtualBox:~$ sudo nano /usr/local/bin/behavior_analyzer.py
[sudo] password for tamam:
tamam@tamam-VirtualBox:~$ python3 /usr/local/bin/behavior_analyzer.py
[*] Behavioral Analyzer Started...
[NORMAL] 10.0.2.4 uploaded normal file: test.png
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: virus.exe
```

The behavior analyzer classifies virus.exe as suspicious because it is an executable file.

Figure 48: virus.exe Log Entry

```
tamam@tamam-VirtualBox:~$ tail -f /var/log/upload_behavior.log
2026-05-16 15:24:06,10.0.2.4,LAN,test.png,png,13
2026-05-16 15:25:48,10.0.2.4,LAN,virus.exe,exe,14
```

The log records the virus.exe upload from the source, documenting the event for later analysis.

Figure 49: PHP File Upload

```
(kali@kali)-[~]
$ for i in {1..6}; do echo "file$i" > file$i.php; curl -X POST -F "file=@file$i.php" http://10.0.2.15:5000/upload
; done
Uploaded file1.php successfully.
Uploaded file2.php successfully.
Uploaded file3.php successfully.
Uploaded file4.php successfully.
Uploaded file5.php successfully.
Uploaded file6.php successfully.
```

A PHP file is uploaded to test detection of script files that may be abused in attacks.

Figure 50: Dangerous PHP Upload Detection

```
tamam@tamam-VirtualBox:~$ sudo nano /usr/local/bin/behavior_analyzer.py
[sudo] password for tamam:
tamam@tamam-VirtualBox:~$ python3 /usr/local/bin/behavior_analyzer.py
[*] Behavioral Analyzer Started...
[NORMAL] 10.0.2.4 uploaded normal file: test.png
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: virus.exe
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file1.php
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file2.php
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file3.php
[ALERT] Excessive uploads detected from 10.0.2.4
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file4.php
[ALERT] Excessive uploads detected from 10.0.2.4
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file5.php
[ALERT] Excessive uploads detected from 10.0.2.4
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file6.php
[ALERT] Excessive uploads detected from 10.0.2.4
```

The analyzer alerts that the PHP file is dangerous or suspicious and detects it successfully.

Figure 51: Upload Activity Log

```
tamam@tamam-VirtualBox:~$ tail -f /var/log/upload_behavior.log
2026-05-16 15:24:06,10.0.2.4,LAN,test.png,png,13
2026-05-16 15:25:48,10.0.2.4,LAN,virus.exe,exe,14
2026-05-16 15:27:20,10.0.2.4,LAN,file1.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file2.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file3.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file4.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file5.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file6.php,php,6
```

The log lists multiple normal and suspicious uploads to document user and file activity.

Figure 52: Large PDF Upload

```
(kali@kali)-[~]
$ dd if=/dev/zero of=big.pdf bs=1M count=6
6+0 records in
6+0 records out
6291456 bytes (6.3 MB, 6.0 MiB) copied, 0.00542406 s, 1.2 GB/s

(kali@kali)-[~]
$ curl -X POST -F "file=@big.pdf" http://10.0.2.15:5000/upload
Uploaded big.pdf successfully.

(kali@kali)-[~]
$
```

A large PDF file is uploaded to test detection of oversized file uploads.

Figure 53: Multi-Upload Analysis

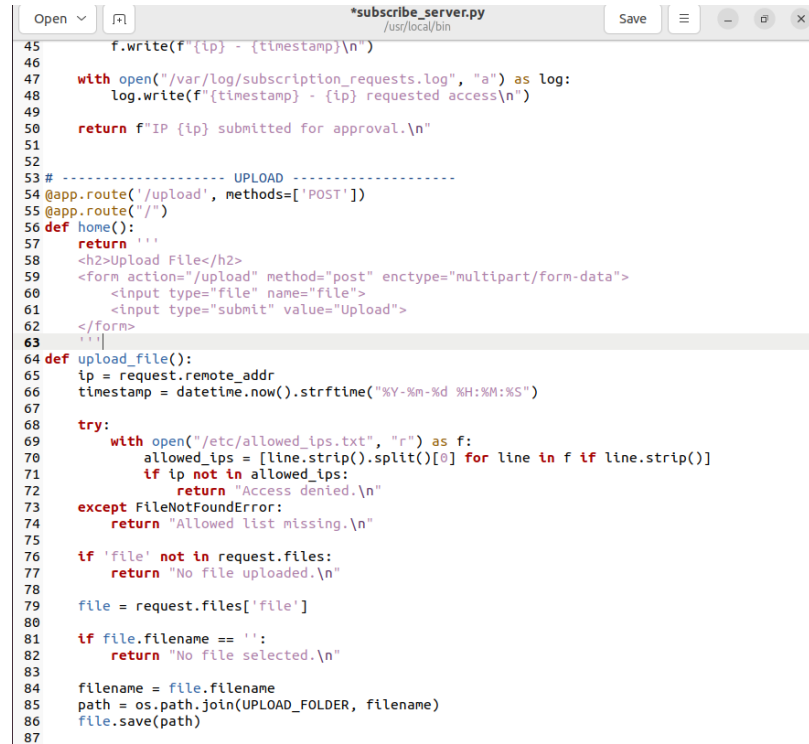
```
tamam@tamam-VirtualBox:~$ sudo nano /usr/local/bin/behavior_analyzer.py
[sudo] password for tamam:
tamam@tamam-VirtualBox:~$ python3 /usr/local/bin/behavior_analyzer.py
[*] Behavioral Analyzer Started...
[NORMAL] 10.0.2.4 uploaded normal file: test.png
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: virus.exe
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file1.php
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file2.php
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file3.php
[ALERT] Excessive uploads detected from 10.0.2.4
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file4.php
[ALERT] Excessive uploads detected from 10.0.2.4
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file5.php
[ALERT] Excessive uploads detected from 10.0.2.4
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file6.php
[ALERT] Excessive uploads detected from 10.0.2.4
[NORMAL] 10.0.2.4 uploaded normal file: big.pdf
[ALERT] Excessive uploads detected from 10.0.2.4
[ALERT] Large file uploaded by 10.0.2.4
```

The analyzer reports multiple alerts, including dangerous files, large files, and repeated upload attempts.

Figure 54: Detailed File Upload Log

```
tamam@tamam-VirtualBox:~$ tail -f /var/log/upload_behavior.log
2026-05-16 15:24:06,10.0.2.4,LAN,test.png,png,13
2026-05-16 15:25:48,10.0.2.4,LAN,virus.exe,exe,14
2026-05-16 15:27:20,10.0.2.4,LAN,file1.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file2.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file3.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file4.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file5.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file6.php,php,6
2026-05-16 15:28:35,10.0.2.4,LAN,big.pdf,pdf,6291456
```

The log documents uploaded filenames, sizes, and sources for investigation.

Figure 55: Upload Server Code


```

45     f.write(f"{ip} - {timestamp}\n")
46
47     with open("/var/log/subscription_requests.log", "a") as log:
48         log.write(f"{timestamp} - {ip} requested access\n")
49
50     return f"IP {ip} submitted for approval.\n"
51
52
53 # ----- UPLOAD -----
54 @app.route('/upload', methods=['POST'])
55 @app.route("/")
56 def home():
57     return '''
58     <h2>Upload File</h2>
59     <form action="/upload" method="post" enctype="multipart/form-data">
60         <input type="file" name="file">
61         <input type="submit" value="Upload">
62     </form>
63 '''
64 def upload_file():
65     ip = request.remote_addr
66     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
67
68     try:
69         with open("/etc/allowed_ips.txt", "r") as f:
70             allowed_ips = [line.strip().split()[0] for line in f if line.strip()]
71             if ip not in allowed_ips:
72                 return "Access denied.\n"
73     except FileNotFoundError:
74         return "Allowed list missing.\n"
75
76     if 'file' not in request.files:
77         return "No file uploaded.\n"
78
79     file = request.files['file']
80
81     if file.filename == '':
82         return "No file selected.\n"
83
84     filename = file.filename
85     path = os.path.join(UPLOAD_FOLDER, filename)
86     file.save(path)
87

```

English: The Flask upload server code is shown, receiving files and writing their information to the log.

Figure 56: File Saving Logic

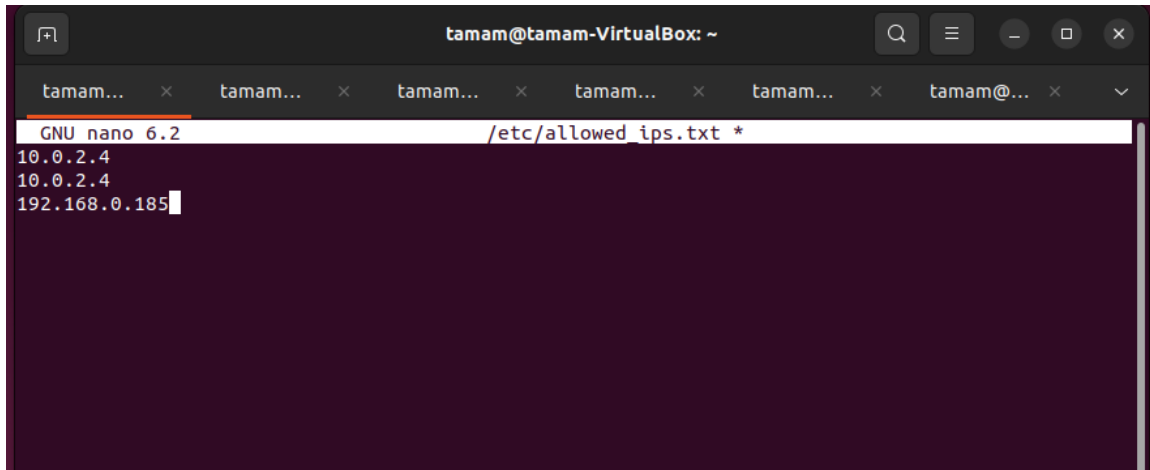

```

45     f.write(f"{ip} - {timestamp}\n")
46
47     with open("/var/log/subscription_requests.log", "a") as log:
48         log.write(f"{timestamp} - {ip} requested access\n")
49
50     return f"IP {ip} submitted for approval.\n"
51
52 # ----- HOME PAGE -----
53
54 @app.route("/")
55 def home():
56     return '''
57     <h2>Upload File</h2>
58     <form action="/upload" method="post" enctype="multipart/form-data">
59         <input type="file" name="file">
60         <input type="submit" value="Upload">
61     </form>
62 '''
63
64 # ----- UPLOAD -----
65 @app.route('/upload', methods=['POST'])
66 def upload_file():
67     ip = request.remote_addr
68     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
69
70     try:
71         with open("/etc/allowed_ips.txt", "r") as f:
72             allowed_ips = [line.strip().split()[0] for line in f if line.strip()]
73             if ip not in allowed_ips:
74                 return "Access denied.\n"
75     except FileNotFoundError:
76         return "Allowed list missing.\n"
77
78     if 'file' not in request.files:

```

The code validates, saves, and logs the uploaded file with filename, size, and source IP.

Figure 57: IP Block/Allow File

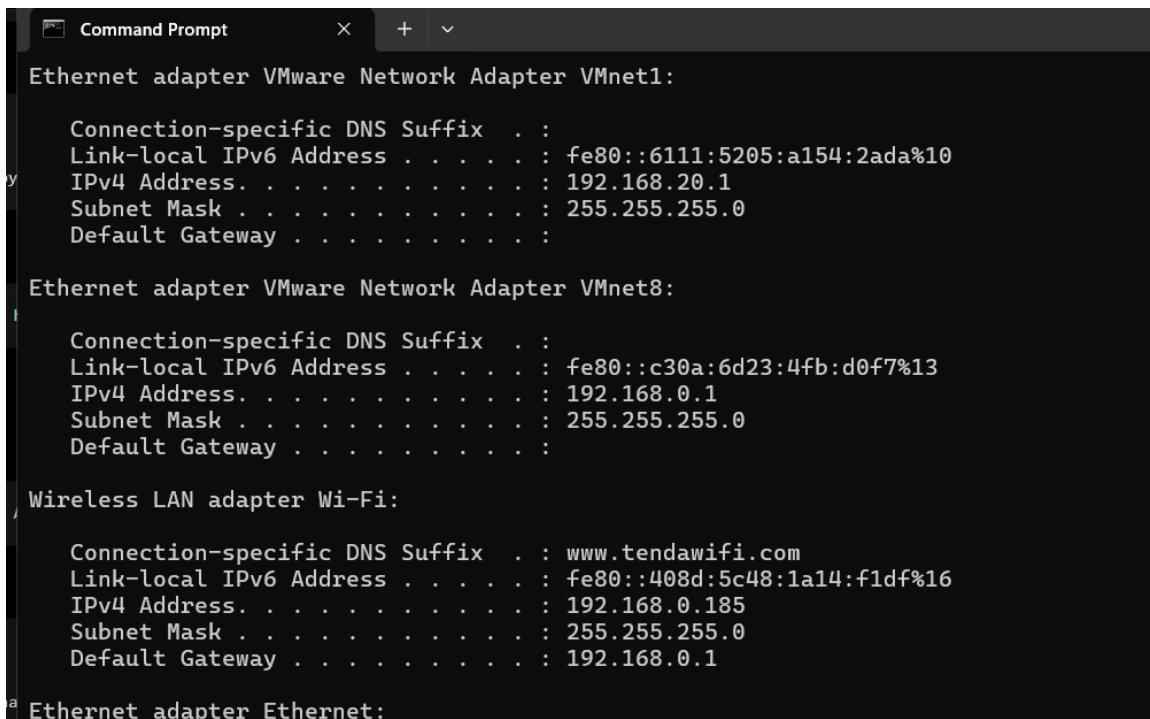


```

tamam@tamam-VirtualBox: ~
GNU nano 6.2 /etc/allowed_ips.txt *
10.0.2.4
10.0.2.4
192.168.0.185
  
```

A file/list containing IP addresses used by the blocking or tracking mechanism is displayed.

Figure 58: Windows Network Information



```

Command Prompt
Ethernet adapter VMware Network Adapter VMnet1:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::6111:5205:a154:2ada%10
    IPv4 Address. . . . . : 192.168.20.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

Ethernet adapter VMware Network Adapter VMnet8:

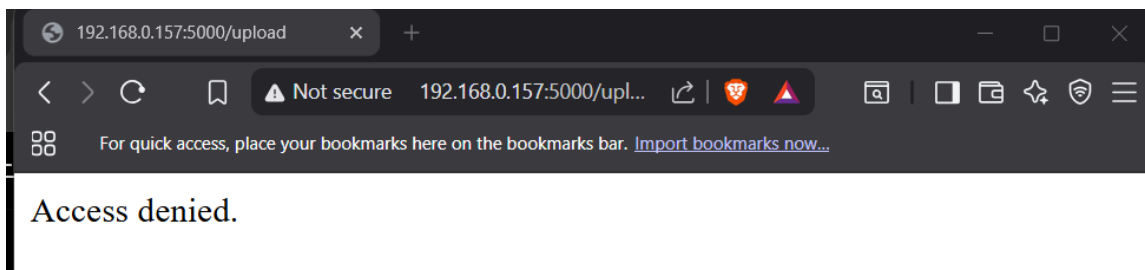
    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::c30a:6d23:4fb:d0f7%13
    IPv4 Address. . . . . : 192.168.0.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

Wireless LAN adapter Wi-Fi:

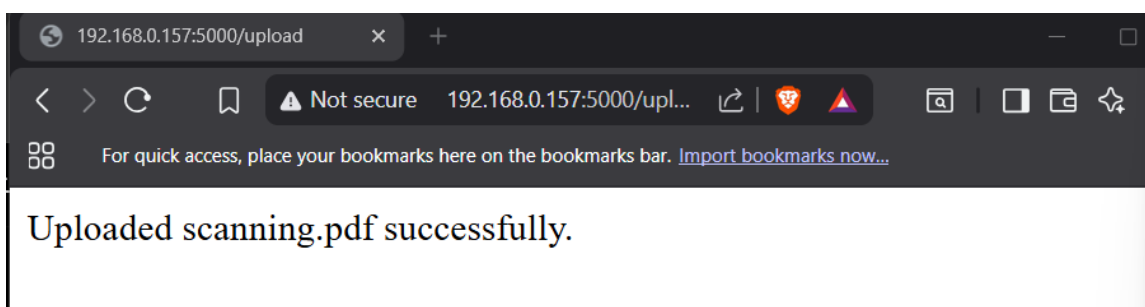
    Connection-specific DNS Suffix  . : www.tendawifi.com
    Link-local IPv6 Address . . . . . : fe80::408d:5c48:1a14:f1df%16
    IPv4 Address. . . . . : 192.168.0.185
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.0.1

Ethernet adapter Ethernet:
  
```

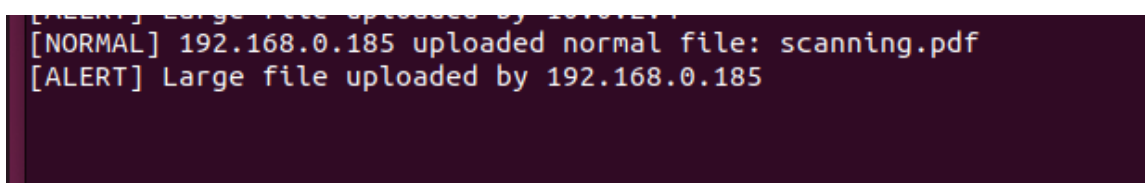
ipconfig shows the Windows client network address within the 192.168.0.0/24 test network.

Figure 59: Browser Access Denied

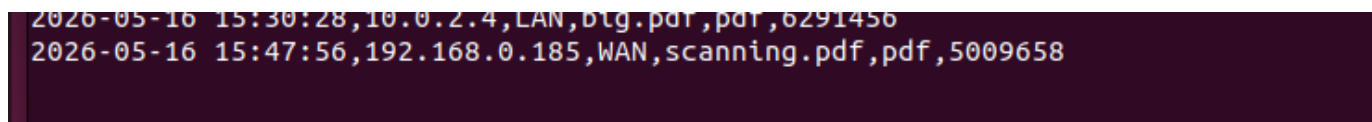
The browser shows Access denied, indicating access control or blocking under the tested condition.

Figure 60: Browser Upload: scanning.pdf

scanning.pdf is uploaded successfully through the web interface, testing browser-to-server upload flow.

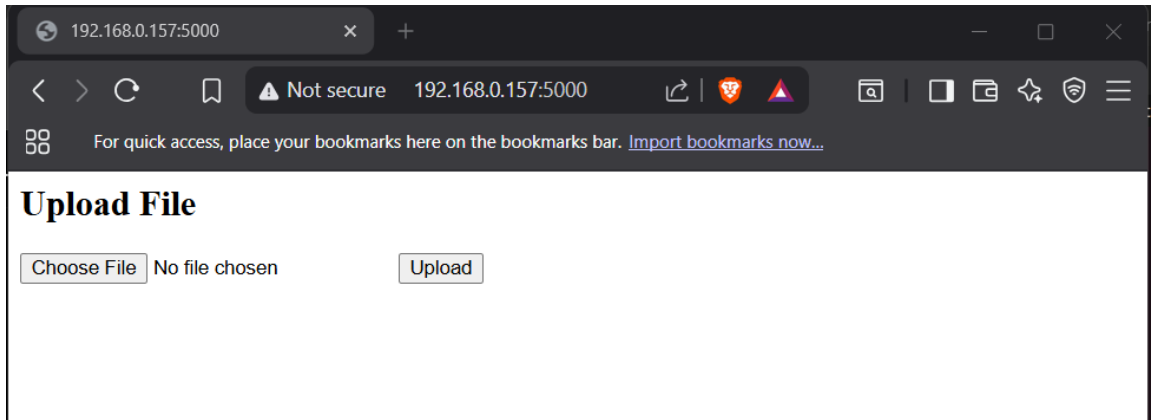
Figure 61: Large File Alert

The analyzer raises an alert indicating that scanning.pdf is a large file uploaded from the client.

Figure 62: Large File Log Entry

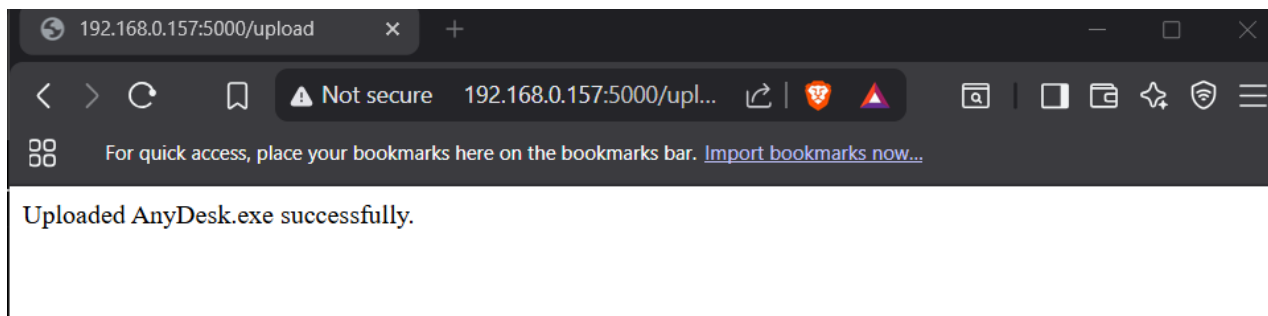
The log records the scanning.pdf upload with its file size for severity assessment.

Figure 63: File Upload Web Interface



The page displays the file upload form used in the application-layer tests.

Figure 64: AnyDesk.exe Upload



AnyDesk.exe is uploaded through the browser to test executable file detection.

Figure 65: Dangerous Executable Alert

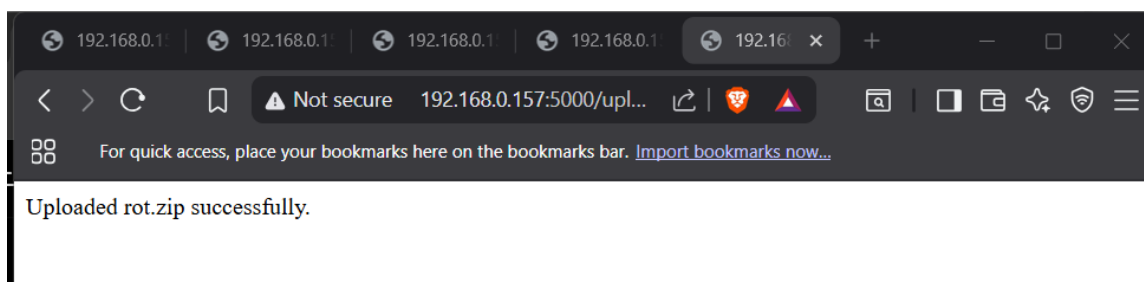
```
[SUSPICIOUS] Dangerous file uploaded by 192.168.0.185: AnyDesk.exe  
[ALERT] Large file uploaded by 192.168.0.185
```

The analyzer alerts that AnyDesk.exe is dangerous because it is an executable file.

Figure 66: AnyDesk.exe Log Entry

```
2026-05-16 15:47:58,192.168.0.185,WAN,Scanning.pdf,pdf,5009058
2026-05-16 15:52:00,192.168.0.185,WAN,AnyDesk.exe,exe,8009144
```

The log records the executable filename, source, and size after upload.

Figure 67: Script File Upload

A script file is uploaded through the browser to test detection of potentially exploitable files.

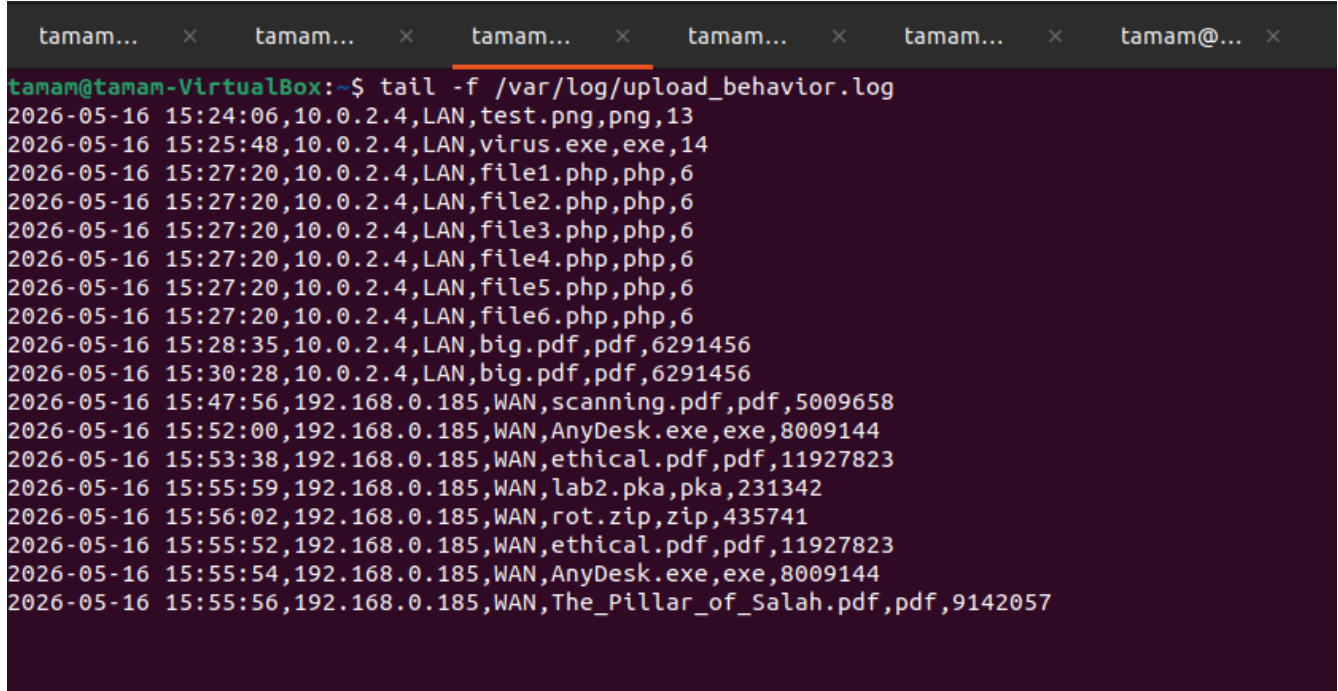
Figure 68: Comprehensive Upload Analysis

```
tamam@tamam-VirtualBox:~$ sudo nano /usr/local/bin/behavior_analyzer.py
[sudo] password for tamam:
tamam@tamam-VirtualBox:~$ python3 /usr/local/bin/behavior_analyzer.py
[*] Behavioral Analyzer Started...
[NORMAL] 10.0.2.4 uploaded normal file: test.png
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: virus.exe
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file1.php
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file2.php
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file3.php
[ALERT] Excessive uploads detected from 10.0.2.4
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file4.php
[ALERT] Excessive uploads detected from 10.0.2.4
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file5.php
[ALERT] Excessive uploads detected from 10.0.2.4
[SUSPICIOUS] Dangerous file uploaded by 10.0.2.4: file6.php
[ALERT] Excessive uploads detected from 10.0.2.4
[NORMAL] 10.0.2.4 uploaded normal file: big.pdf
[ALERT] Excessive uploads detected from 10.0.2.4
[ALERT] Large file uploaded by 10.0.2.4
[NORMAL] 10.0.2.4 uploaded normal file: big.pdf
[ALERT] Excessive uploads detected from 10.0.2.4
[ALERT] Large file uploaded by 10.0.2.4
[NORMAL] 192.168.0.185 uploaded normal file: scanning.pdf
[ALERT] Large file uploaded by 192.168.0.185
[SUSPICIOUS] Dangerous file uploaded by 192.168.0.185: AnyDesk.exe
[ALERT] Large file uploaded by 192.168.0.185
[NORMAL] 192.168.0.185 uploaded normal file: ethical.pdf
[ALERT] Large file uploaded by 192.168.0.185
[ALERT] Excessive uploads detected from 192.168.0.185
[NORMAL] 192.168.0.185 uploaded normal file: ethical.pdf
[ALERT] Excessive uploads detected from 192.168.0.185
[ALERT] Large file uploaded by 192.168.0.185
[SUSPICIOUS] Dangerous file uploaded by 192.168.0.185: AnyDesk.exe
[ALERT] Excessive uploads detected from 192.168.0.185
[ALERT] Large file uploaded by 192.168.0.185
[NORMAL] 192.168.0.185 uploaded normal file: The_Pillar_of_Salah.pdf
[ALERT] Excessive uploads detected from 192.168.0.185
[ALERT] Large file uploaded by 192.168.0.185
```

The analyzer reports several alerts including dangerous files, repeated attempts, and email alert sending.

7. Email Alerts and Threat Scoring

Figure 69: Comprehensive Behavior Log



```
tamam@tamam-VirtualBox:~$ tail -f /var/log/upload_behavior.log
2026-05-16 15:24:06,10.0.2.4,LAN,test.png,png,13
2026-05-16 15:25:48,10.0.2.4,LAN,virus.exe,exe,14
2026-05-16 15:27:20,10.0.2.4,LAN,file1.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file2.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file3.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file4.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file5.php,php,6
2026-05-16 15:27:20,10.0.2.4,LAN,file6.php,php,6
2026-05-16 15:28:35,10.0.2.4,LAN,big.pdf,pdf,6291456
2026-05-16 15:30:28,10.0.2.4,LAN,big.pdf,pdf,6291456
2026-05-16 15:47:56,192.168.0.185,WAN,scanning.pdf,pdf,5009658
2026-05-16 15:52:00,192.168.0.185,WAN,AnyDesk.exe,exe,8009144
2026-05-16 15:53:38,192.168.0.185,WAN,ethical.pdf,pdf,11927823
2026-05-16 15:55:59,192.168.0.185,WAN,lab2.pka,pka,231342
2026-05-16 15:56:02,192.168.0.185,WAN,rot.zip,zip,435741
2026-05-16 15:55:52,192.168.0.185,WAN,ethical.pdf,pdf,11927823
2026-05-16 15:55:54,192.168.0.185,WAN,AnyDesk.exe,exe,8009144
2026-05-16 15:55:56,192.168.0.185,WAN,The_Pillar_of_Salah.pdf,pdf,9142057
```

global_behavior.log lists multiple upload activities from different IPs with filenames and sizes.

Figure 70: Analyzer Monitoring Stream

```

tamam@tamam-VirtualBox:~$ python3 /usr/local/bin/behavior_analyzer.py
[*] Threat Scoring Analyzer Started...
2026-05-16 16:23:10,10.0.2.4,LAN,test.png,Normal File Upload,+0,0,Normal
2026-05-16 16:23:10,10.0.2.4,LAN,virus.exe,Malicious File Upload,+3,3,Suspicious
2026-05-16 16:23:10,10.0.2.4,LAN,file1.php,Malicious File Upload,+3,6,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,file2.php,Malicious File Upload,+3,9,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,file3.php,Malicious File Upload,+3,12,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,file3.php,Repeated Upload Attempts,+2,14,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,file4.php,Malicious File Upload,+3,17,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,file4.php,Repeated Upload Attempts,+2,19,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,file5.php,Malicious File Upload,+3,22,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,file5.php,Repeated Upload Attempts,+2,24,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,file6.php,Malicious File Upload,+3,27,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,file6.php,Repeated Upload Attempts,+2,29,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,big.pdf,Normal File Upload,+0,29,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,big.pdf,Repeated Upload Attempts,+2,31,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,big.pdf,Large File Upload,+2,33,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,big.pdf,Normal File Upload,+0,33,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,big.pdf,Repeated Upload Attempts,+2,35,High Threat
2026-05-16 16:23:10,10.0.2.4,LAN,big.pdf,Large File Upload,+2,37,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,scanning.pdf,Normal File Upload,+0,0,Normal
2026-05-16 16:23:10,192.168.0.185,WAN,scanning.pdf,Large File Upload,+2,2,Suspicious
2026-05-16 16:23:10,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,+3,5,Medium Threat
2026-05-16 16:23:10,192.168.0.185,WAN,AnyDesk.exe,Large File Upload,+2,7,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,ethical.pdf,Normal File Upload,+0,7,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,ethical.pdf,Large File Upload,+2,9,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,lab2.pka,Unknown File Type,+1,10,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,rot.zip,Suspicious Archive Upload,+1,11,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,rot.zip,Repeated Upload Attempts,+2,13,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,ethical.pdf,Normal File Upload,+0,13,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,ethical.pdf,Repeated Upload Attempts,+2,15,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,ethical.pdf,Large File Upload,+2,17,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,+3,20,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,AnyDesk.exe,Repeated Upload Attempts,+2,22,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,AnyDesk.exe,Large File Upload,+2,24,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,The_Pillar_of_Salah.pdf,Normal File Upload,+0,24,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,The_Pillar_of_Salah.pdf,Repeated Upload Attempts,+2,26,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,The_Pillar_of_Salah.pdf,Large File Upload,+2,28,High Threat

```

The analyzer continuously monitors incoming events and classifies them during the experiment.

Figure 71: Attack Timeline Log

```

at
2026-05-16 16:23:10,192.168.0.185,WAN,The_Pillar_of_Salah.pdf,Repeated Upload Attempts,+2,26,High Threat
2026-05-16 16:23:10,192.168.0.185,WAN,The_Pillar_of_Salah.pdf,Large File Upload,+2,28,High Threat
2026-05-16 16:23:10,192.168.0.197,WAN,10 AnyCast.pdf,Normal File Upload,+0,0,Normal
2026-05-16 16:23:10,192.168.0.197,WAN,10 AnyCast.pdf,Normal File Upload,+0,0,Normal
2026-05-16 16:23:10,192.168.0.197,WAN,WAF document.docx,Unknown File Type,+1,1,Normal
2026-05-16 16:23:10,192.168.0.197,WAN,WAF document.docx,Unknown File Type,+1,2,Suspicious
2026-05-16 16:23:10,192.168.0.197,WAN,HW4.asm.txt,Unknown File Type,+1,3,Suspicious
2026-05-16 16:23:10,192.168.0.197,WAN,HW4.asm.txt,Repeated Upload Attempts,+2,5,Medium Threat

```

attack_timeline.log records event sequence, such as repeated uploads and threat-level changes over time.

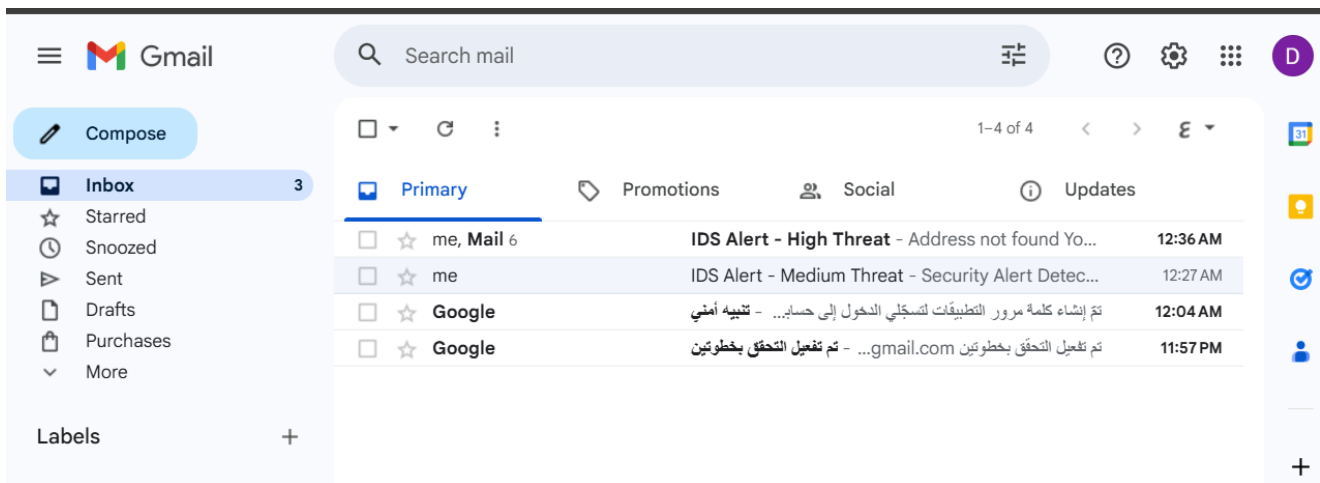
Figure 72: Email Configuration

```

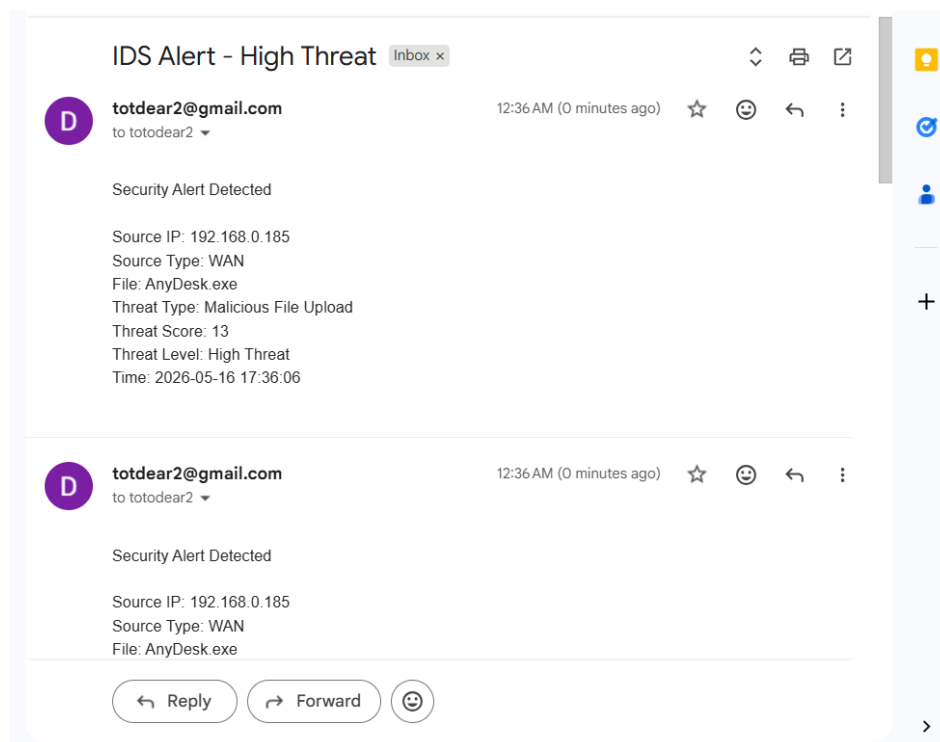
tamam@tamam-VirtualBox: ~
GNU nano 6.2 /etc/ids_email.conf *
EMAIL_SENDER=[REDACTED]
EMAIL_PASSWORD=[REDACTED]
EMAIL_RECEIVER=[REDACTED]

```

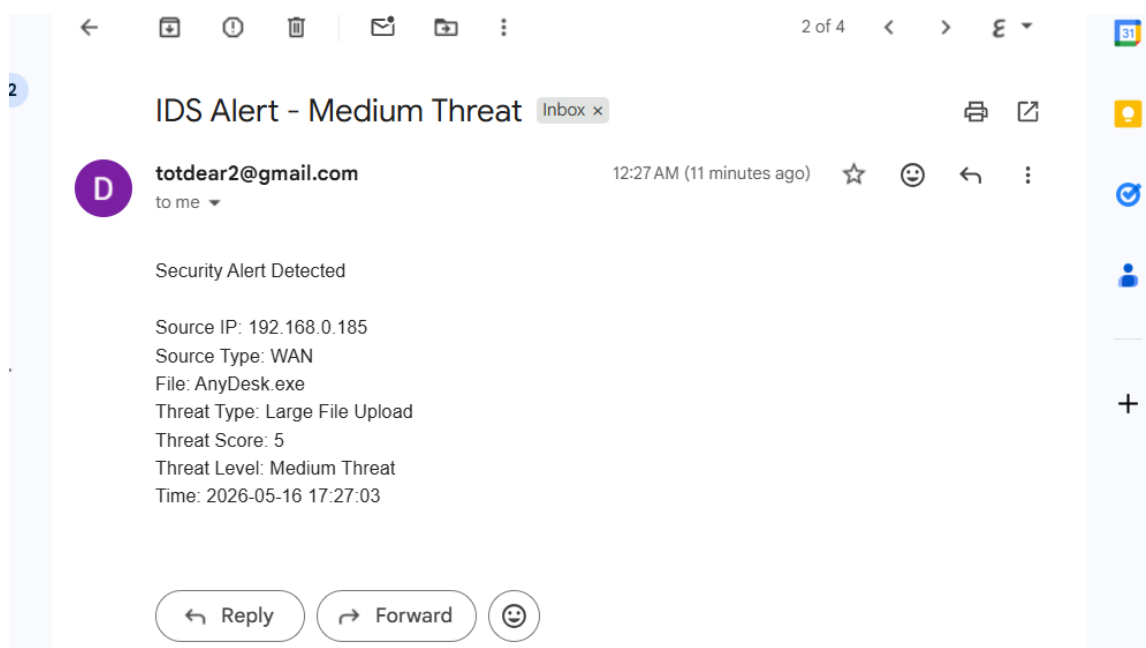
The email configuration file used to send IDS alerts when threats are detected is shown.

Figure 73: IDS Alerts in Gmail

IDS alert messages appear in Gmail, confirming successful alert delivery from the system.

Figure 74: High Threat Email Alert

A high-threat email alert displays the source, threat type, time, and details.

Figure 75: Medium Threat Email Alert

A medium-threat email alert documents a security event with lower severity than High Threat.

Figure 76: Threat Score Calculation

```

tamam@tamam-VirtualBox:~$ sudo pkill -f behavior_analyzer.py
sudo python3 /usr/local/bin/behavior_analyzer.py
[*] Threat Scoring Analyzer Started...
2026-05-16 17:32:45,192.168.0.185,WAN,16.jpg,Normal File Upload,+0,0,Normal
2026-05-16 17:32:45,192.168.0.185,WAN,16.jpg,Normal File Upload,+0,0,Normal
2026-05-16 17:32:45,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,+3,3,Suspicious
2026-05-16 17:32:45,192.168.0.185,WAN,AnyDesk.exe,Large File Upload,+2,5,Medium Threat
[+] Email alert sent.
2026-05-16 17:33:23,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,+3,8,High Threat
[+] Email alert sent.
2026-05-16 17:33:24,192.168.0.185,WAN,AnyDesk.exe,Large File Upload,+2,10,High Threat
[+] Email alert sent.
2026-05-16 17:36:06,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,+3,13,High Threat
[+] Email alert sent.
2026-05-16 17:36:07,192.168.0.185,WAN,AnyDesk.exe,Repeated Upload Attempts,+2,15,High Threat
[+] Email alert sent.
2026-05-16 17:36:09,192.168.0.185,WAN,AnyDesk.exe,Large File Upload,+2,17,High Threat
[+] Email alert sent.

```

The analyzer calculates threat scores, raises severity, and sends an alert when risk becomes significant.

Figure 77: IP Blocking Script

```

tamam@... x  tamam@... x  tamam@... x  tamam@... x  tamam@... x  tamam@... x
GNU nano 6.2 /usr/local/bin/block_ip.sh *
#!/bin/bash

IP=$1

iptables -A INPUT -s $IP -j DROP

echo "$(date) BLOCKED $IP" >> /var/log/blocked_ips.log

```

block_ip.sh uses iptables to add a DROP rule and block a suspicious IP address.

Figure 78: IPS Blocking Triggered

```

tamam@tamam-VirtualBox:~$ sudo python3 /usr/local/bin/behavior_analyzer.py
[*] Threat Scoring Analyzer Started...
2026-05-16 18:07:23,192.168.0.185,WAN,16.jpg,Normal File Upload,+0,0,Normal
2026-05-16 18:07:23,192.168.0.185,WAN,16.jpg,Normal File Upload,+0,0,Normal
2026-05-16 18:07:23,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,+3,3,Suspicious
2026-05-16 18:07:23,192.168.0.185,WAN,AnyDesk.exe,Large File Upload,+2,5,Medium Threat
[+] Email alert sent.
2026-05-16 18:07:26,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,+3,8,High Threat
[+] Email alert sent.
[BLOCKED] 192.168.0.185 has been blocked by firewall.
2026-05-16 18:07:27,192.168.0.185,WAN,AnyDesk.exe,Large File Upload,+2,10,High Threat
[+] Email alert sent.
2026-05-16 18:07:29,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,+3,13,High Threat
[+] Email alert sent.
2026-05-16 18:07:31,192.168.0.185,WAN,AnyDesk.exe,Repeated Upload Attempts,+2,15,High Threat
[+] Email alert sent.
2026-05-16 18:07:33,192.168.0.185,WAN,AnyDesk.exe,Large File Upload,+2,17,High Threat
[+] Email alert sent.

```

The analyzer detects a malicious file and triggers firewall blocking.

8. IPS Blocking and Firewall Validation

Figure 79: Making the Blocking Script Executable

```

tamam@tamam-VirtualBox:~$ sudo nano /usr/local/bin/block_ip.sh
[sudo] password for tamam:
tamam@tamam-VirtualBox:~$ sudo chmod +x /usr/local/bin/block_ip.sh

```

chmod is used to make block_ip.sh executable so it can be called automatically.

Figure 80: iptables Before Blocking

```

tamam@tamam-VirtualBox:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

```

iptables is shown before blocking, with the default policy still set to ACCEPT.

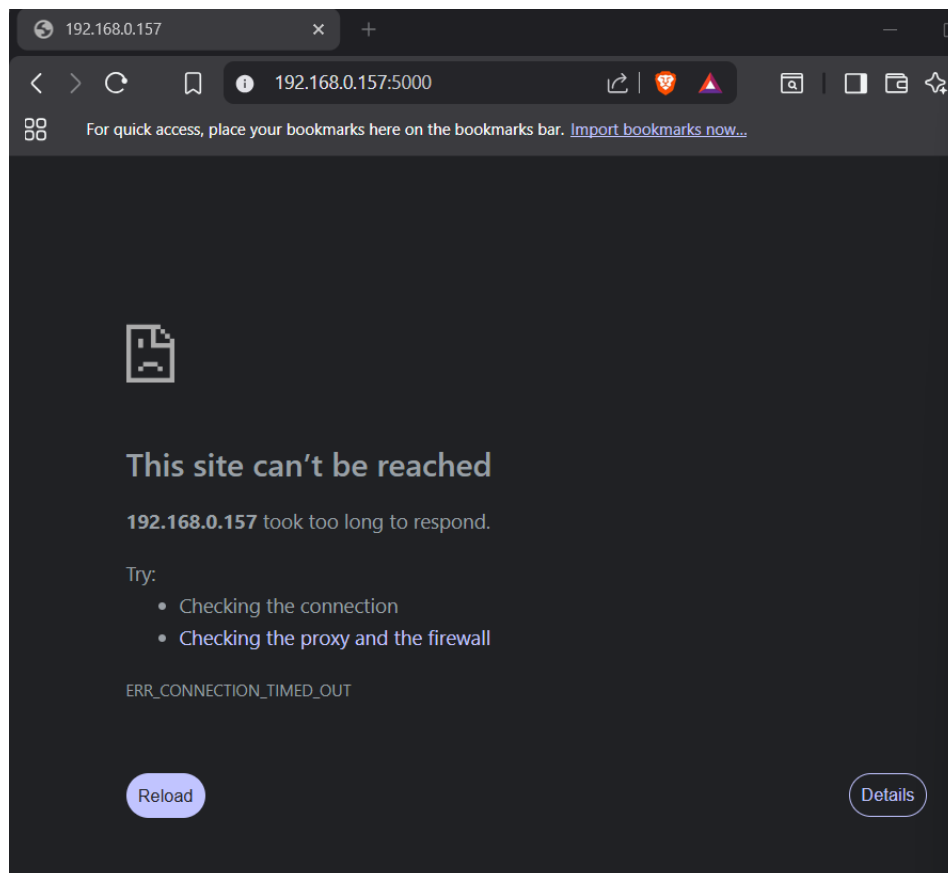
Figure 81: iptables After Blocking

```
tamam@tamam-VirtualBox:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP      all  --  192.168.0.185         anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
tamam@tamam-VirtualBox:~$
```

A DROP rule for 192.168.0.185 appears, confirming that the IPS function is active.

Figure 82: Access Failure After Blocking

The browser cannot reach the site after the client IP is blocked.

Figure 83: Blocking Log

```
tamam@tamam-VirtualBox:~$ tail -f /var/log/upload_behavior.log
2026-05-16 17:22:44,192.168.0.185,WAN,16.jpg,jpg,197556
2026-05-16 17:23:06,192.168.0.185,WAN,16.jpg,jpg,197556
2026-05-16 17:24:40,192.168.0.185,WAN,AnyDesk.exe,exe,8009144
2026-05-16 17:33:18,192.168.0.185,WAN,AnyDesk.exe,exe,8009144
2026-05-16 17:36:00,192.168.0.185,WAN,AnyDesk.exe,exe,8009144
```

The log documents suspicious uploads and the IP blocking status performed by the IPS.

Figure 84: IP Block Confirmation

```
[41]+ Stopped                  sudo python3 /usr/local/bin/behavior_analyzer.py
tamam@tamam-VirtualBox:~$ sudo python3 /usr/local/bin/behavior_analyzer.py
[*] Threat Scoring Analyzer Started...
2026-05-16 18:18:17,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,+3,3,Suspicious
2026-05-16 18:18:17,192.168.0.185,WAN,AnyDesk.exe,Large File Upload,+2,5,Medium Threat
[+] Email alert sent.
2026-05-16 18:19:09,192.168.0.185,WAN,rot.zip,Suspicious Archive Upload,+1,6,High Threat
[+] Email alert sent.
[BLOCKED] 192.168.0.185 has been blocked by firewall.
```

The analyzer confirms that an IP was blocked due to suspicious upload activity.

Figure 85: Alert Email List

1-8 of 8

Primary

Promotions

Social

Updates

☐	☒	me, Mail 2	New	IDS Alert - High Threat - Address not found Yo...	1:19 AM
☐	☒	me, Mail 2	New	IDS Alert - Medium Threat - Address not foun...	1:18 AM
☐	☒	me, Mail 10	New	IDS Alert - High Threat - Address not found Yo...	1:07 AM
☐	☒	me, Mail 2	New	IDS Alert - Medium Threat - Address not foun...	1:07 AM
☐	☒	me, Mail 6		IDS Alert - High Threat - Address not found Your...	12:36 AM
☐	☒	me		IDS Alert - Medium Threat - Security Alert Detec...	12:27 AM
☐	☒	Google		تم إنشاء كلمة مرور التطبيقات لتسجيلي الدخول إلى حسابي... - تنبيه أمني	12:04 AM
☐	☒	Google		تم تفعيل التحقق بخطوتين ...gmail.com - تم تفعيل التحقق بخطوتين	11:57 PM

0% of 15 GB used

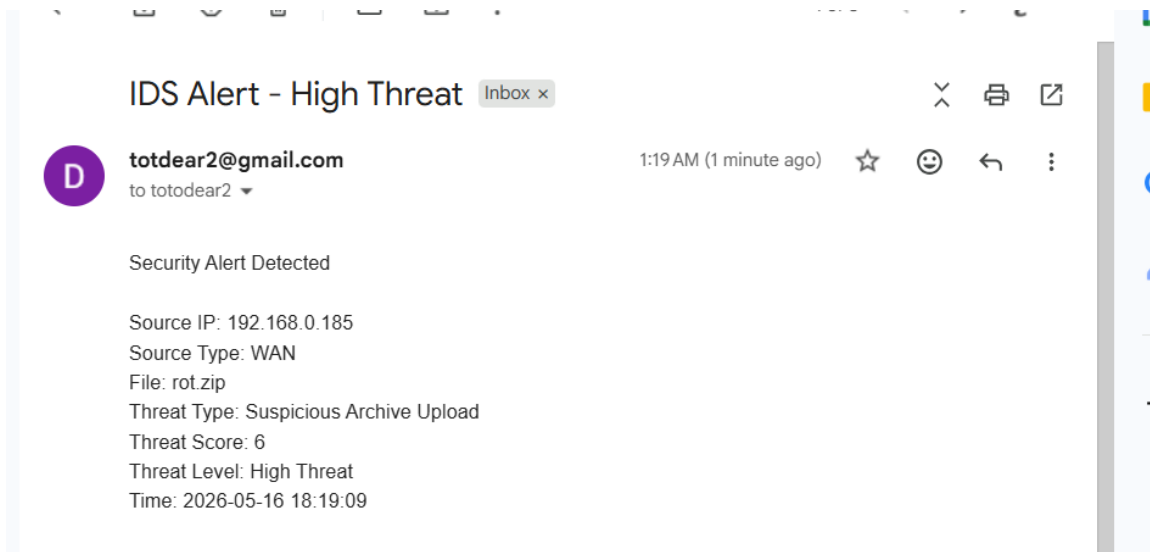
[Terms](#) · [Privacy](#) · [Program Policies](#)

Last account activity: 0 minutes ago

Open in 1 other location · [Details](#)

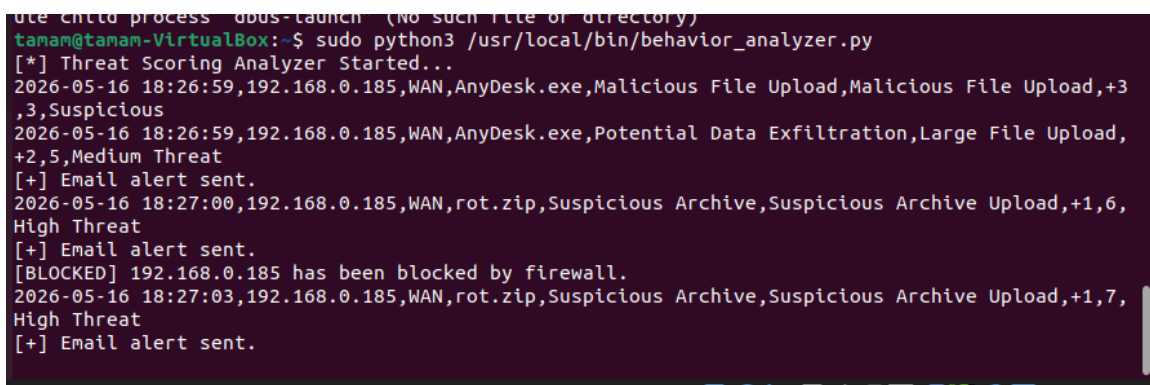
Gmail shows multiple IDS alert emails with different severities, proving continuous notification capability.

Figure 86: High Threat Alert Details



The email shows details of a high-threat alert related to a suspicious file upload.

Figure 87: Advanced Alert Analysis



The analyzer handles multiple scenarios such as malicious and repeated uploads and sends alerts.

Figure 88: Suspicious Archive Detection

```

tamam@tamam-VirtualBox:~$ sudo python3 /usr/local/bin/behavior_analyzer.py
[*] Threat Scoring Analyzer Started...
2026-05-16 18:30:55,192.168.0.185,WAN,routing.zip,Suspicious Archive,Suspicious Archive Upload,+
1,1,Normal
2026-05-16 18:31:07,192.168.0.185,WAN,routing.zip,Suspicious Archive,Suspicious Archive Upload,+
1,2,Suspicious
2026-05-16 18:31:15,192.168.0.185,WAN,routing.zip,Suspicious Archive,Suspicious Archive Upload,+
1,3,Suspicious
2026-05-16 18:31:45,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,Malicious File Upload,+3
,6,High Threat
[+] Email alert sent.
[BLOCKED] 192.168.0.185 has been blocked by firewall.
2026-05-16 18:31:47,192.168.0.185,WAN,AnyDesk.exe,Potential Data Exfiltration,Large File Upload,
+2,8,High Threat
[+] Email alert sent.

```

The analyzer detects a suspicious archive and potential data exfiltration based on behavior.

Figure 89: Preparing Attack Timeline Log

```

target      port  opt  source      destination
tamam@tamam-VirtualBox:~$ sudo touch /var/log/attack_timeline.log
sudo chmod 666 /var/log/attack_timeline.log
tamam@tamam-VirtualBox:~$

```

attack_timeline.log is created or permissioned to store the sequence of security events.

Figure 90: Analyzer, Blocking, and Timeline Integration

```

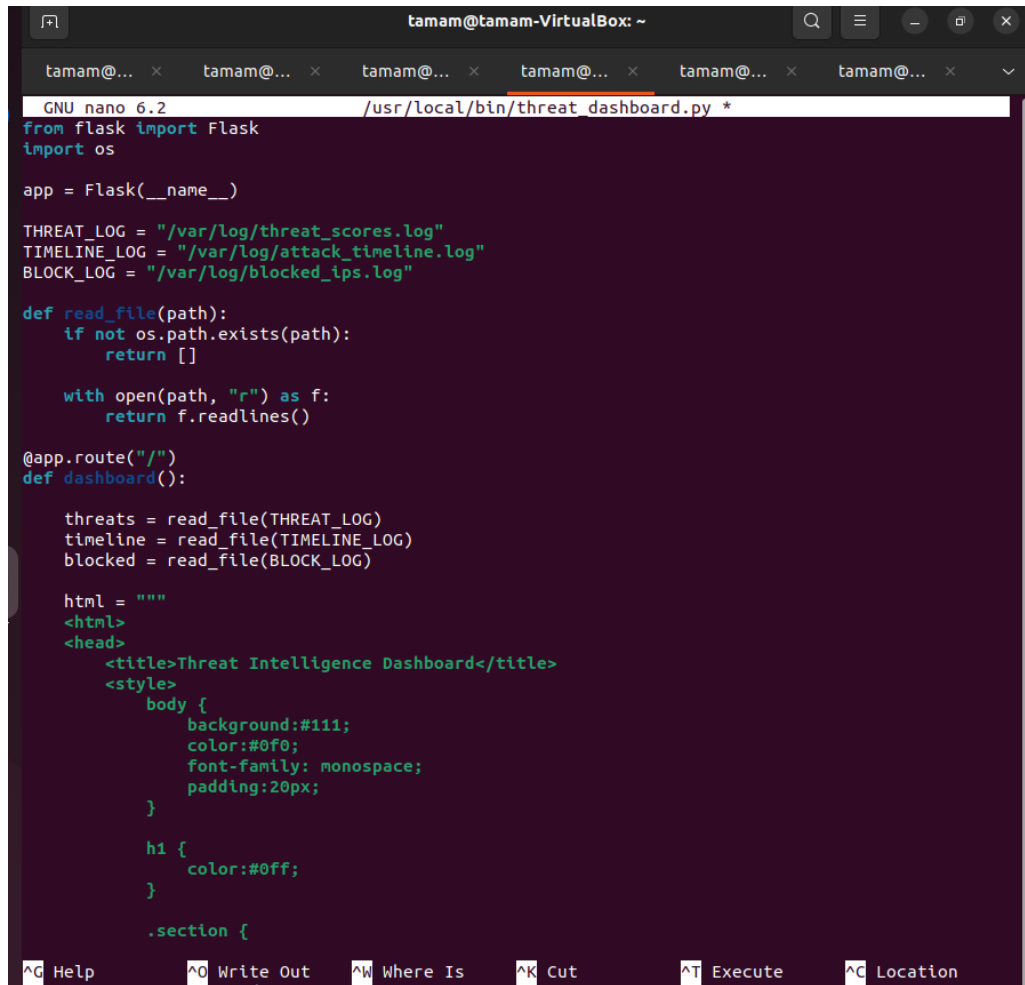
tamam@tamam-VirtualBox:~$ sudo python3 -m py_compile /usr/local/bin/behavior_analyzer.py
tamam@tamam-VirtualBox:~$ sudo python3 /usr/local/bin/behavior_analyzer.py
[*] Threat Scoring Analyzer Started...
[TIMELINE] 2026-05-16 18:36:54 | 192.168.0.185 | Suspicious Archive Upload detected
[TIMELINE] 2026-05-16 18:36:54 | 192.168.0.185 | Threat level changed to Normal
2026-05-16 18:36:54,192.168.0.185,WAN,routing.zip,Suspicious Archive,Suspicious Archive Upload,+
1,1,Normal
[TIMELINE] 2026-05-16 18:36:54 | 192.168.0.185 | Suspicious Archive Upload detected
[TIMELINE] 2026-05-16 18:36:54 | 192.168.0.185 | Threat level changed to Suspicious
2026-05-16 18:36:54,192.168.0.185,WAN,routing.zip,Suspicious Archive,Suspicious Archive Upload,+
1,2,Suspicious
[TIMELINE] 2026-05-16 18:36:54 | 192.168.0.185 | Suspicious Archive Upload detected
[TIMELINE] 2026-05-16 18:36:54 | 192.168.0.185 | Threat level changed to Suspicious
2026-05-16 18:36:54,192.168.0.185,WAN,routing.zip,Suspicious Archive,Suspicious Archive Upload,+
1,3,Suspicious
[TIMELINE] 2026-05-16 18:36:54 | 192.168.0.185 | Malicious File Upload detected
[TIMELINE] 2026-05-16 18:36:54 | 192.168.0.185 | Threat level changed to High Threat
2026-05-16 18:36:54,192.168.0.185,WAN,AnyDesk.exe,Malicious File Upload,Malicious File Upload,+3
,6,High Threat
[+] Email alert sent.
[BLOCKED] 192.168.0.185 has been blocked by firewall.
[TIMELINE] 2026-05-16 18:36:56 | 192.168.0.185 | IP blocked by IPS
[TIMELINE] 2026-05-16 18:36:56 | 192.168.0.185 | Large File Upload detected
[TIMELINE] 2026-05-16 18:36:56 | 192.168.0.185 | Threat level changed to High Threat
2026-05-16 18:36:56,192.168.0.185,WAN,AnyDesk.exe,Potential Data Exfiltration,Large File Upload,
+2,8,High Threat
[+] Email alert sent.

```

The analyzer shows integrated events: detection, severity update, blocking, and timeline logging.

9. Threat Intelligence Dashboard

Figure 91: Dashboard Code



```

tamam@tamam-VirtualBox: ~
GNU nano 6.2 /usr/local/bin/threat_dashboard.py *
from flask import Flask
import os

app = Flask(__name__)

THREAT_LOG = "/var/log/threat_scores.log"
TIMELINE_LOG = "/var/log/attack_timeline.log"
BLOCK_LOG = "/var/log/blocked_ips.log"

def read_file(path):
    if not os.path.exists(path):
        return []

    with open(path, "r") as f:
        return f.readlines()

@app.route("/")
def dashboard():

    threats = read_file(THREAT_LOG)
    timeline = read_file(TIMELINE_LOG)
    blocked = read_file(BLOCK_LOG)

    html = """
    <html>
    <head>
        <title>Threat Intelligence Dashboard</title>
        <style>
            body {
                background:#111;
                color:#0f0;
                font-family: monospace;
                padding:20px;
            }

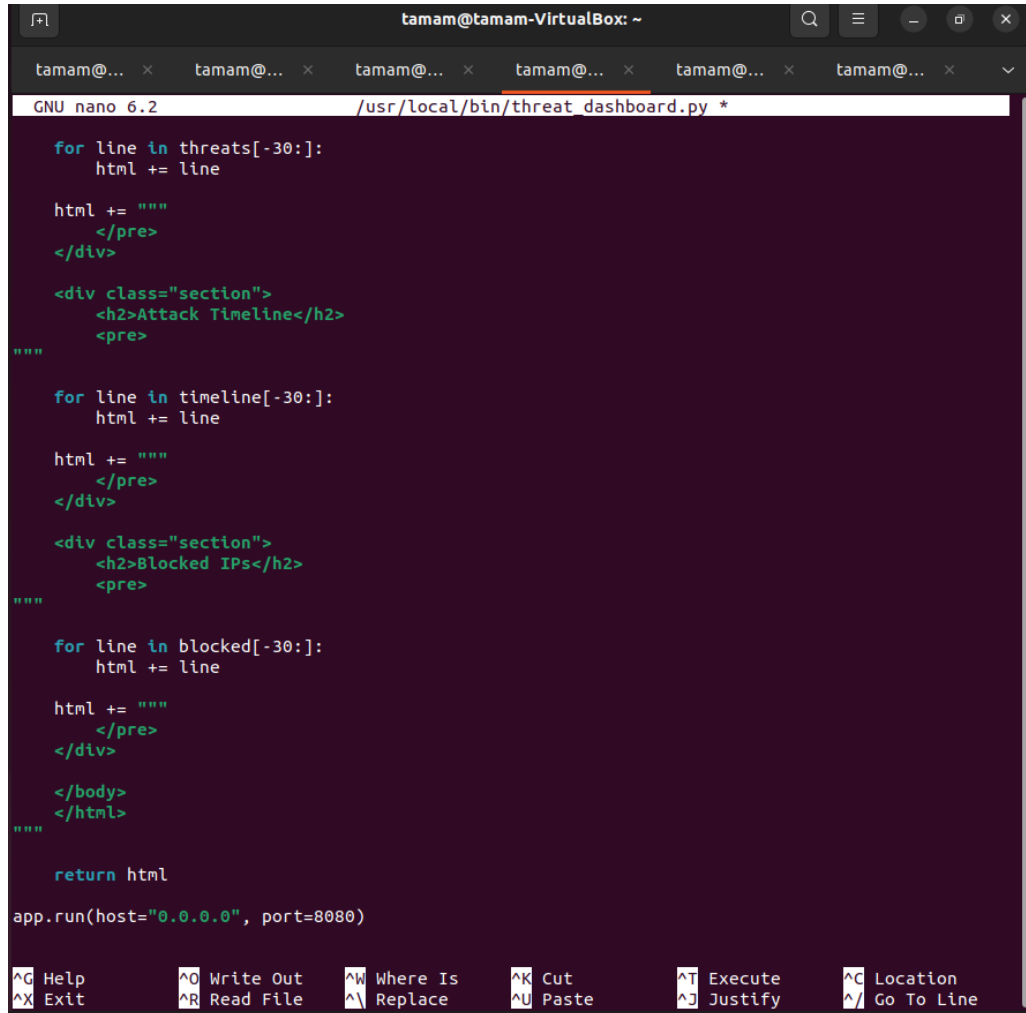
            h1 {
                color:#0ff;
            }

            .section {

```

Part of the Flask dashboard code reads logs and prepares threat intelligence data.

Figure 92: Dashboard Template



```

tamam@tamam-VirtualBox: ~
GNU nano 6.2 /usr/local/bin/threat_dashboard.py *

for line in threats[-30:]:
    html += line

html += """
</pre>
</div>

<div class="section">
<h2>Attack Timeline</h2>
<pre>
"""

for line in timeline[-30:]:
    html += line

html += """
</pre>
</div>

<div class="section">
<h2>Blocked IPs</h2>
<pre>
"""

for line in blocked[-30:]:
    html += line

html += """
</pre>
</div>

</body>
</html>
"""

return html

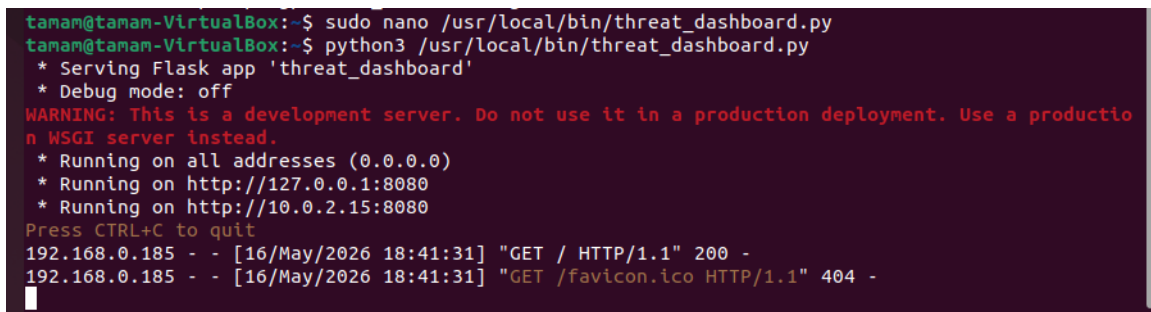
app.run(host="0.0.0.0", port=8080)

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line

```

Part of the HTML template displays logs, threats, and blocked IP addresses.

Figure 93: Running the Threat Dashboard



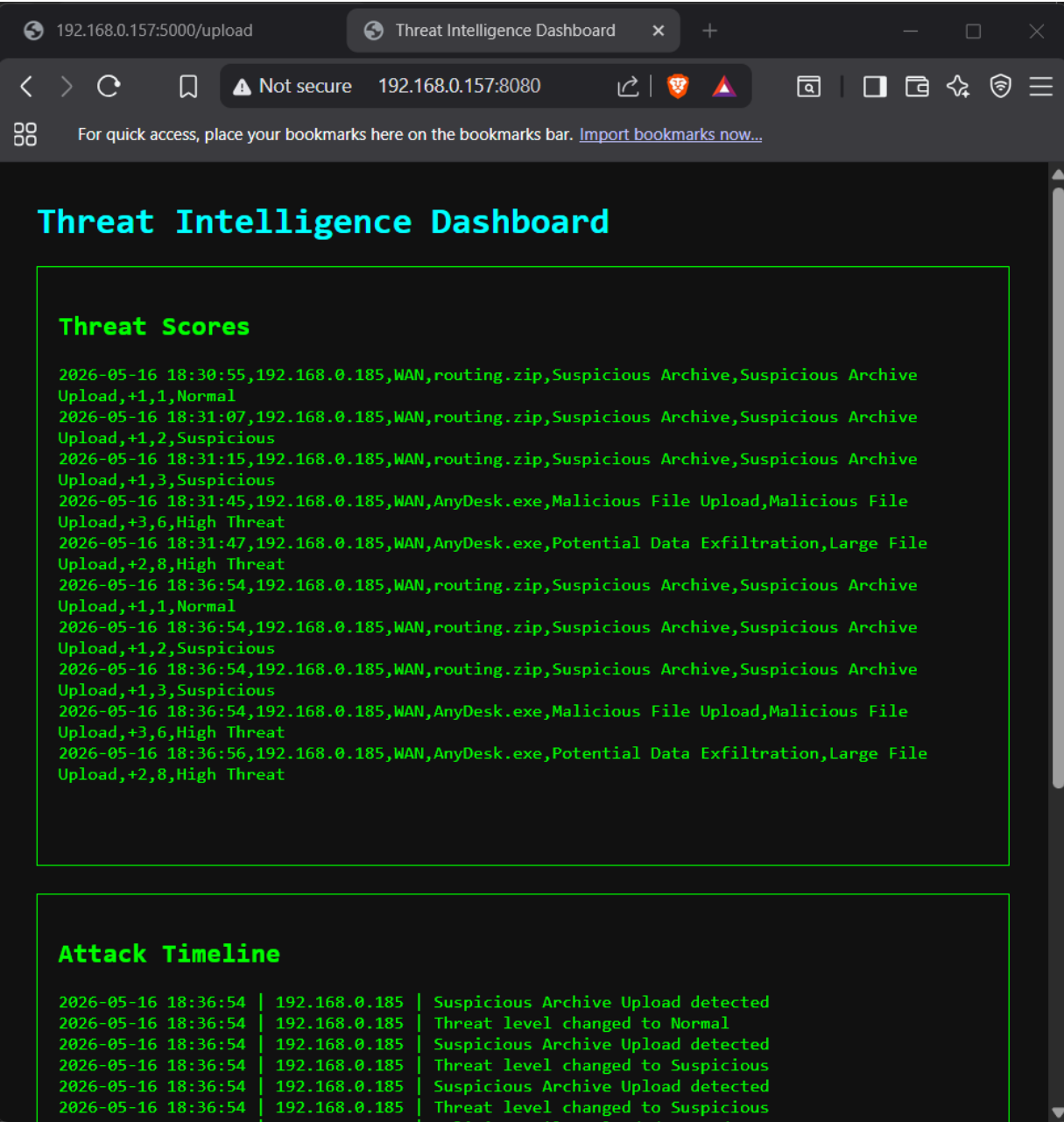
```

tamam@tamam-VirtualBox:~$ sudo nano /usr/local/bin/threat_dashboard.py
tamam@tamam-VirtualBox:~$ python3 /usr/local/bin/threat_dashboard.py
* Serving Flask app 'threat_dashboard'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://10.0.2.15:8080
Press CTRL+C to quit
192.168.0.185 - - [16/May/2026 18:41:31] "GET / HTTP/1.1" 200 -
192.168.0.185 - - [16/May/2026 18:41:31] "GET /favicon.ico HTTP/1.1" 404 -

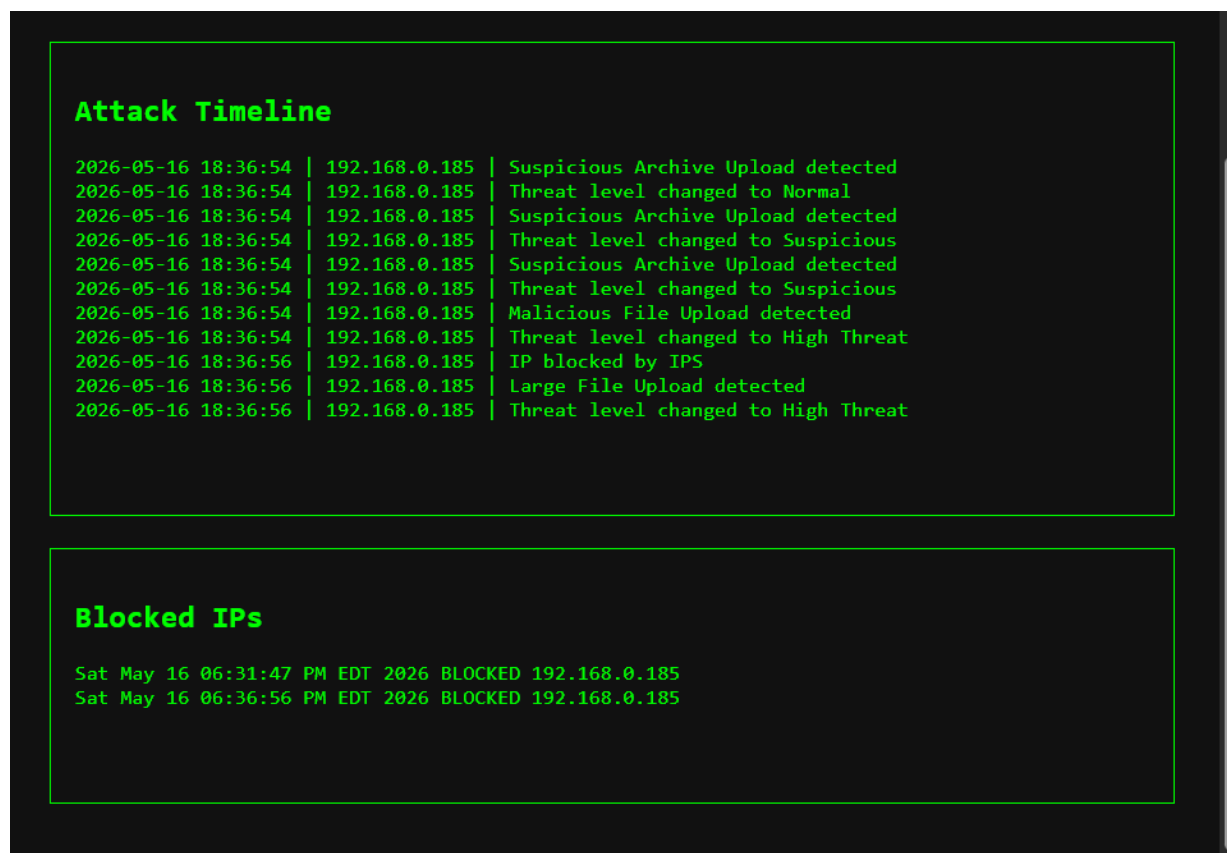
```

threat_dashboard.py is started and a Flask server is opened to display results in the browser.

Figure 94: Threat Intelligence Dashboard



The dashboard presents threat summaries and event logs in a centralized readable interface.

Figure 95: Timeline and Blocked IPs

The dashboard shows the Attack Timeline and Blocked IPs list, demonstrating IDS/IPS integration.

10. Final Conclusion

The results show that the platform successfully monitored multiple interfaces separately, ran Suricata with custom rules, detected ICMP/HTTP and repeated traffic, and analyzed file uploads with threat classification.

The integration between IDS and IPS was also demonstrated through email alerts, suspicious IP blocking using iptables, and final event visualization through the Threat Intelligence Dashboard.